UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS DE CRATEÚS
CURSO DE GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

GUSTAVO CAMPELO DE SOUSA

FERRAMENTA MULTIAGENTE INTEGRADA COM LLM OPEN-SOURCE PARA APOIO À CORREÇÃO DE TCCS DE ESTUDANTES DE GRADUAÇÃO

CRATEÚS
2025

GUSTAVO CAMPELO DE SOUSA

FERRAMENTA MULTIAGENTE INTEGRADA COM LLM OPEN-SOURCE PARA APOIO À CORREÇÃO DE TCCS DE ESTUDANTES DE GRADUAÇÃO

Trabalho de Conclusão de Curso 1 apresentado ao Curso de Graduação em Ciência da Computação da Universidade Federal do Ceará, como requisito parcial à obtenção do grau de bacharel em Ciência da Computação.

Orientador: Prof. Dr. José Wellington Franco da Silva

Coorientador: Prof. Dr. Antonio Emerson Barros Tomaz

CRATEÚS

2025

GUSTAVO CAMPELO DE SOUSA

FERRAMENTA MULTIAGENTE INTEGRADA COM LLM OPEN-SOURCE PARA APOIO À CORREÇÃO DE TCCS DE ESTUDANTES DE GRADUAÇÃO

Trabalho de Conclusão de Curso 1 apresentado ao Curso de Graduação em Ciência da Computação da Universidade Federal do Ceará, como requisito parcial à obtenção do grau de bacharel em Ciência da Computação.

Aprovada em:

BANCA EXAMINADORA

Prof. Dr. José Wellington Franco da Silva (Orientador)
Universidade Federal do Ceará (UFC)

Prof. Dr. Antonio Emerson Barros Tomaz (Coorientador)
Universidade Federal do Ceará (UFC)

Prof. Dr. Carlos de Oliveira Caminha
Universidade Federal do Ceará (UFC)

Prof. Me. Marciel Barros Pereira
Universidade Federal do Ceará (UFC)

À minha mãe e a todos que me auxiliam, mesmo que não saibam, a fugir da mediocridade que acomete a mim e a vida do brasileiro em uma pátria entorpecida pela indiferença, porém não destituída do labor de quem tenta reduzi-la.

AGRADECIMENTOS

Agradeço a meus familiares, em especial a minha mãe Antonia e a minha irmã Sophia, que todos os dias fizeram presentes em meus pensamentos e almejares mais sublimes, motivando-me a não desistir e a conseguir, no futuro, fornecer um futuro melhor para ambas.

Aos grandes escritores da literatura, que respeito tremendamente, particularmente ao Professor J. R. R. Tolkien, que me fez apaixonar pela fantasia, a Machado de Assis e José de Alencar, que reacenderam meu interesse pela fantástica e apaixonante literatura brasileira. E, para chegar onde estou, nota-se que sempre estive apoiando-me nos ombros de gigantes.

Aos amigos e colegas com quem convivi ao longo dessa epopeia, mostrando-me a unicidade de seres que um mesmo espaço é capaz de receber e moldar.

Ao meu grande amigo Gabriel Falcão, por todas as conversas e discussões que tivemos nessa primeira década de amizade, e quando esta foi retomada em 2018, passei a vê-lo como um amigo mentor, que, mesmo sem perceber, me ensinou a ter uma percepção das coisas que outras pessoas não adquirem, mesmo que parem para pensar durante vários dias.

Aos professores da Universidade Federal do Ceará por todo o conhecimento transmitido ao longo de tantos anos, em especial Wellington Franco e Emerson Tomaz, meus orientadores, e também aos professores Rafael Martins Barros e Rennan Ferreira Dantas, que ministraram as disciplinas mais encantadoras que a computação já viu. Aos professores Bruno Riccelli e Marciel Barros, que sempre me foram solícitos e possuem aquele espírito do educador e a ambição de desenvolver projetos grandiosos em prol da comunidade acadêmica.

E aos meus companheiros do EngineLab, pelo cotidiano, as conversas corriqueiras e as risadas mais sinceras já arrancadas, seja lá por qual motivo for.

E a mim, por não ter sucumbido à miséria intelectual da qual o brasileiro está sujeito todos os dias, contaminado, como já dizia José de Alencar em sua carta ao Dr. Jaguaribe: "Já estava eu meio descrido das cousas, e mais dos homens; e por isso buscava na literatura diversão à tristeza que me infundia o estado da pátria entorpecida pela indiferença". A indiferença me incumbe, mas não sucumbo, estou aqui e agradeço por isso.

“Vereis amor da pátria, não movido
De prêmio vil, mas alto e quase eterno:
Que não é prêmio vil ser conhecido
Por um pregão do ninho meu paterno.
Ouvi: vereis o nome engrandecido
Daqueles de quem sois senhor superno,
E julgareis qual é mais excelente,
Se ser do mundo Rei, se de tal gente.“

(Luiz Vaz de Camões)

RESUMO

A correção do Trabalho de Conclusão de Curso (TCC) é uma etapa crucial na formação da pesquisa do aluno de graduação, no entanto, esse processo pode ser cansativo tanto para o aluno em sua pesquisa quanto para o orientador durante o acompanhamento, devido a fatores como sobrecarga de tarefas e retornos pouco específicos sobre o conteúdo da pesquisa. Este trabalho desenvolverá uma ferramenta de análise e correção de TCCs composta por três agentes especializados: correção gramatical, encadeamento lógico e rigor metodológico, integrando um *Large Language Model* (LLM) *open-source* orientado por engenharia de *prompt*. Para o conjunto de dados deste estudo, serão extraídos TCCs presentes no repositório institucional de produções científicas da Universidade Federal de Ceará (UFC). Para alcançar o objetivo proposto, apresenta-se um desenho metodológico que visa selecionar um LLM *open-source* que suporte entrada e saída de textos em português e compará-lo a um modelo proprietário, por meio de um *baseline* comparativo entre as abordagens, visto que, em limitações de tamanho e valor monetário, os modelos *open-source* se saem melhor em sua escolha. A ferramenta, chamada *Academic Review Agents for Methodological Improvements* (ARAMIS), receberá o texto do aluno e, somada a um *prompt* especializado, será processada pelo LLM e retornará uma revisão estruturada baseada nas diretrizes definidas na configuração dos três agentes. A avaliação da ferramenta será realizada por meio da métrica *System Usability Scale* (SUS), permitindo uma análise imparcial e confiável por parte dos usuários que testarão o sistema. O objetivo desta pesquisa é verificar se a ferramenta consegue diagnosticar e orientar a construção de encadeamentos lógicos e rigor metodológico nos TCCs. Os resultados obtidos podem direcionar os alunos a reduzir o tempo de desenvolvimento da pesquisa científica, com retornos específicos e direcionados abrangidos pelo ARAMIS, levando-os a adotar ferramentas desse estilo, garantindo, porém, que estas serão utilizadas como complemento no processo, sem substituir o papel humano.

Palavras-chave: Feedback. TCC. LLM. Open-source. ARAMIS. Engenharia de prompt. SUS

ABSTRACT

The correction of Undergraduate Final Projects is a crucial step in the formation and continuity of undergraduate student research, however, this process can be tiring for both the student in their research and the advisor during the monitoring process, due to factors such as task overload and unspecific feedback on the research content. This work will develop a tool for analyzing and correcting final projects composed of three specialized agents: grammatical correction, logical flow, and methodological rigor, integrating an open-source Large Language Model (LLM) guided by prompt engineering. For the dataset of this study, final projects will be extracted from the institutional repository of scientific productions of the Federal University of Ceará (UFC). To achieve the proposed objective, a methodological design is presented that aims to select an open-source LLM that supports input and output of texts in portuguese and compare it to a proprietary model, through a comparative baseline between the approaches, since in size and monetary limitations, open-source models perform better in their choice. The tool, called Academic Review Agents for Methodological Improvements (ARAMIS), will receive the student's text, and combined with a specialized prompt, will be processed by the LLM and will return a structured review based on the guidelines defined in the configuration of the three agents. The tool evaluation will be performed using the System Usability Scale (SUS) metric, allowing an impartial and reliable analysis by the users who will test the system. The objective of this research is to verify whether the tool can diagnose and guide the construction of logical flows and methodological rigor in Final Projects. The results obtained can direct students to reduce the development time of scientific research, with specific and targeted feedback covered by ARAMIS, leading them to adopt tools of this style, while ensuring that they will be used as a complement in the process, without replacing the human role.

Keywords: Feedback. Final Project. LLM. Open-source. ARAMIS. Prompt Engineering. SUS

LISTA DE FIGURAS

Figura 1 – Exemplo do *few-shot learning* para classificação de sentimento utilizando métodos baseados em *prompts*. 26
Figura 2 – Exemplo de *Chain-of-Thought* 27
Figura 3 – Exemplo do questionário SUS que será utilizado 39
Figura 4 – Exemplo do cálculo realizado na avaliação SUS 40
Figura 5 – A figura apresenta o fluxograma do ARAMIS, detalhando suas principais etapas. 42

LISTA DE TABELAS

Tabela 1 – Comparação das principais contribuições dos trabalhos relacionados 35
Tabela 2 – Cronograma das próximas atividades 48

LISTA DE ABREVIATURAS E SIGLAS

AM	Aprendizado de Máquina
AP	Aprendizado Profundo
ARAMIS	*Academic Review Agents for Methodological Improvements*
GPT	*Generative Pre-Trained Transformers*
GPU	*Graphics Processing Unit*
HTTP	*Hypertext Transfer Protocol*
IA	Inteligência Artificial
ICLR	*International Conference on Learning Representations*
JSON	*JavaScript Object Notation*
LLM	*Large Language Model*
MARG	*Multi-Agent Review Generator*
MARG-S	*Multi-Agent Review Generator with Specialized Agents*
ML	*Machine Learning*
MVP	*Minumum Viable Product*
NeurIPS	*Conference on Neural Information Processing Systems*
NLP	*Natural Language Processing*
OECD	*Organisation for Economic Co-operation and Development*
PDF	*Portable Document Format*
PISA	*Programme for International Student Assessment*
PLN	Processamento de Linguagem Natural
RN	Redes Neurais
SI	Sistemas de Informação
SUS	*System Usability Scale*
SWIF2T	*Scientific WrIting Focused Feedback Tool*
TCC	Trabalho de Conclusão de Curso
UFC	Universidade Federal de Ceará
UFPE	Universidade Federal de Pernambuco
YAML	*YAML Ain't Markup Language*

SUMÁRIO

1 INTRODUÇÃO 13
1.1 Declaração do problema e questão de pesquisa 14
1.2 Justificativa 15
1.3 Objetivo geral 16
1.4 Objetivos específicos 16
1.5 Estrutura do trabalho 17
2 FUNDAMENTAÇÃO TEÓRICA 18
2.1 Trabalho de conclusão de curso 18
2.2 Inteligência artificial 19
2.3 Aprendizado de máquina 21
2.3.1 Aprendizado supervisionado 22
2.3.2 Aprendizado não supervisionado 22
2.3.3 Aprendizado por reforço 22
2.4 Processamento de linguagem natural 23
2.5 Modelos de linguagem de grande escala 24
2.6 Engenharia de prompt 25
2.6.1 Few-Shot prompting 26
2.6.2 Chain-of-Thought prompting 26
3 TRABALHOS RELACIONADOS 28
3.1 Can large language models provide useful feedback on research papers? A large-scale empirical analysis 28
3.2 Automated focused feedback generation for scientific writing assistance 29
3.3 MARG: Multi-Agent review generation for scientific papers 30
3.4 OpenReviewer: a specialized large language model for generating critical scientific paper reviews 31
3.5 Análise comparativa 32
4 PROCEDIMENTO METODOLÓGICO 36
4.1 Tecnologias de implementação da aplicação 36
4.2 Conjunto de dados 37
4.3 Avaliação da ferramenta 38

4.3.1 *Cálculo do SUS* 39
5 **PROPOSTA** 41
5.1 **ARAMIS** 41
5.1.1 *Arquitetura e fluxo do ARAMIS* 42
5.2 **Escolha do LLM open-source** 43
5.3 **Tamanho do modelo** 44
5.4 **Função dos agentes do ARAMIS** 44
5.5 **Modelagem dos prompts** 46
6 **CONSIDERAÇÕES FINAIS E CRONOGRAMA** 48
REFERÊNCIAS 49

1 INTRODUÇÃO

O Trabalho de Conclusão de Curso (TCC) constitui a etapa final do percurso acadêmico dos estudantes de graduação, sendo um requisito indispensável na maioria das Universidades para a obtenção do grau de bacharel ou licenciado na respectiva área de formação. Em geral, a etapa imediatamente anterior à obtenção do título acadêmico é a elaboração do TCC, que se refere à pesquisa e à redação de um documento que atenda aos critérios acadêmicos vigentes. Assim, de acordo com Carboni e Nogueira (2008), o TCC, além de introduzir o aluno na pesquisa por meio de recursos metodológicos, o capacitará a resolver problemas e incentivará a prosseguir na carreira acadêmica.

Ao elaborar um texto dessa importância, é necessária a atenção contínua ao trabalho, pois, no processo de avaliação do TCC, observa-se o rigor no desenvolvimento conforme as normas, o encadeamento lógico das ideias, a profundidade e a apresentação do conhecimento do tema estudado (CARBONI; NOGUEIRA, 2008). Diante disso, a revisão textual, seja pelo aluno e/ou orientador, deixou de se limitar apenas a aspectos normativos e passou a observar aspectos como verossimilhança e encadeamento narrativo (PEREZ; BOENAVIDES, 2017). Nesse processo, o discente possui um papel fundamental na garantia da qualidade, atuando em parceria com o orientador — cuja experiência tende a facilitar a identificação de inconsistências textuais. Contudo, mesmo com o empenho conjunto, a necessidade de sucessivas revisões persiste, gerando retrabalho e consumo excessivo de tempo. Diante dessas dificuldades, fica evidente a necessidade de soluções que agilizem a revisão, como sistemas de Inteligência Artificial (IA) projetados para corrigir textos conforme as particularidades do contexto acadêmico.

No estudo de Srivarsha *et al.* (2025), argumenta-se que a correção automatizada aprimora significativamente a experiência do usuário, reduzindo erros tipográficos e linguísticos no conteúdo escrito. Para isso, são incorporados, segundo Coppin (2004), métodos a computadores para que estes compreendam a linguagem humana, chamadas de técnicas de Processamento de Linguagem Natural (PLN).

As técnicas de PLN são combinadas com modelos de *Deep Learning*, um subconjunto da área de *Machine Learning* (ML), que visa permitir que modelos computacionais compostos por múltiplas camadas de processamento aprendam representações de dados com múltiplos níveis de abstração, auxiliando, inclusive, na detecção de colocações linguísticas inadequadas (LECUN *et al.*, 2015).

Em um estudo anterior, Zhang e Zhang (2023) apresentaram um enfoque mais

específico na precisão da tradução automática para o inglês, embora também vise a redução de erros gramaticais, empregando técnicas de análise semântica e métodos especializados de aumento de dados. Tais trabalhos, embora aplicados a domínios ligeiramente diferentes, convergem para o objetivo comum de melhorar a qualidade do texto por meio da detecção e correção automatizada de erros. Tais técnicas aplicadas podem ser adaptáveis à revisão de textos acadêmicos, como os TCCs, visto que em Lunsford e Lunsford (2008), os principais problemas identificados nesses documentos incluem erros de grafia (13,7% dos casos) e a ausência de vírgulas após elementos introdutórios (9,6%), entre outros.

1.1 Declaração do problema e questão de pesquisa

O valor formativo de um TCC reside, sobretudo, na capacidade de articular ideias com coerência, demonstrar rigor metodológico e avançar um argumento científico claro. Entretanto, levantamentos recentes mostram que os maiores obstáculos dos graduandos já não se restringem à ortografia: numa sondagem com licenciandos da Universidade Federal de Pernambuco (UFPE), as principais dificuldades residem na escrita acadêmica e na escassa prática de pesquisa (MONTENEGRO, 2024). Docentes orientadores relatam que muitos trabalhos carecem de coerência entre tema, problema, objetivos, método e conclusão (MEDEIROS *et al.*, 2015). Os aspectos que se destacam como principais obstáculos à qualidade dos trabalhos incluem dificuldades na seleção e citação de fontes pertinentes, na aplicação do rigor metodológico, a baixa compreensão do método científico e a inabilidade para estruturar o projeto de pesquisa (MATOS; FREITAS, 2020). Embora sistemas de correção automática consigam corrigir grafia e pontuação, nenhuma solução de IA foi ainda treinada em português especificamente para diagnosticar e orientar aspectos macroestruturais, como o encadeamento argumentativo de capítulos ou a adequação metodológica de um trabalho de conclusão de curso. Até o momento, essas avaliações permanecem predominantemente a cargo de orientadores humanos, perpetuando ciclos de retrabalho que atrasam as defesas e reduzem a qualidade científica dos trabalhos. Essa lacuna tecnológica sugere a seguinte questão de pesquisa: *Como sistemas de Inteligência Artificial especializados em textos acadêmicos em português podem diagnosticar e orientar a construção de encadeamentos lógicos e rigor metodológico em Trabalhos de Conclusão de Curso, reduzindo o número de revisões manuais sem comprometer a autoria e a qualidade científica.*

1.2 Justificativa

A exigência de formalidade gramatical e normativa na elaboração de TCCs e artigos científicos requer do aluno competências linguísticas bem desenvolvidas. No entanto, segundo a *Organisation for Economic Co-operation and Development* (OECD), no estudo *The State of Learning and Equity in Education*, estudantes brasileiros de 15 anos de idade obtiveram 410 pontos em leitura no *ranking Programme for International Student Assessment* (PISA), abaixo da média da própria OECD, que é de 476 pontos (OECD, 2023). Esse dado revela deficiências na formação básica, especialmente em leitura e interpretação, habilidades fundamentais para a produção textual no ensino superior.

De acordo com Fagundes *et al.* (2014), a educação recebida nos níveis anteriores impacta diretamente o desempenho dos estudantes no ensino superior, sobretudo nos semestres iniciais. Um dos fatores mais influentes é a experiência educativa prévia, que deve fornecer a base teórica e metodológica necessária para a adaptação do aluno à vida universitária. No entanto, a lacuna revelada pelos dados do PISA aponta para deficiências nessa experiência prévia, afetando a capacidade de análise crítica, o domínio das disciplinas e as técnicas de estudo e elaboração de trabalhos.

Essa deficiência compromete a autonomia dos estudantes para interpretar textos complexos, realizar pesquisas e produzir textos formais, como o TCC e artigos científicos. Assim, os resultados do PISA não apenas indicam um problema na educação básica, mas também antecipam os obstáculos enfrentados por muitos estudantes no ensino superior — o que reforça a necessidade de suporte específico no processo de escrita acadêmica. E toda essa formalidade requerida tende a gerar:

- Exaustão cognitiva e frustração nos alunos;
- Sobrecarga para orientadores, com tempo dedicado a correções de erros muitas vezes superficiais;
- Desvio do foco com mentorias menos essenciais e incisivas;
- Falta de padronização nas correções, resultando em inconsistências avaliativas.

Embora o método tradicional de correção seja funcional e, por vezes, o mais acurado, dada a capacidade humana de decisão, a questão do tempo ainda é um ponto a ser considerado, tornando-se dispendioso para as partes envolvidas. Ao adotar métodos de correção automática especializados antes do envio do texto ao professor orientador, o autor pode dedicar mais tempo à pesquisa e a outras atribuições relevantes, tornando o trabalho mais produtivo e focado nos

resultados esperados. E, ao transferir para a IA a carga da revisão formal, a plataforma permitirá que o tempo poupado seja reinvestido no cerne da pesquisa: aprimoramento teórico, análise crítica dos resultados e a discussão das contribuições científicas.

Esse trabalho justifica a necessidade de criação de uma ferramenta como recurso de suporte ao aluno em sua produção científica, independentemente de sua boa formação prévia na educação básica, contribuindo para o melhor aproveitamento do tempo na escrita do TCC, para o aumento do rigor acadêmico, para a promoção da qualidade textual e a otimização do processo de orientação.

1.3 Objetivo geral

Desenvolver uma ferramenta automatizada de revisão de TCCs, integrada a um *LLM open-source*, que permita receber um *feedback* de correção sobre aspectos textuais e estruturais do trabalho, conforme as normas brasileiras vigentes, melhorando a qualidade da escrita, assegurando coerência lógica e rigor metodológico, e diminuindo o número de revisões enviadas ao orientador.

1.4 Objetivos específicos

- Coletar um corpus de TCCs em português;
- Escolher um LLM *open-source* e um proprietário para a *baseline* de testes;
- Direcionar o LLM com técnicas de engenharia de prompt para aprimorar as correções, utilizando os TCCs do corpus coletado;
- Desenvolver uma abordagem multiagente, onde o modelo *open-source* será direcionado a funções diferentes;
- Implementar uma plataforma com o modelo *open-source* integrado, para enviar o texto ao modelo e receber o *feedback* dele sobre as seções desejadas, com resposta estruturada.
- Avaliar a ferramenta com métricas de avaliação de usabilidade, como o *System Usability Scale* (SUS).

1.5 Estrutura do trabalho

O presente trabalho é composto por seis capítulos, incluindo esta Introdução. O Capítulo 2 aborda o referencial teórico que embasa este estudo. No Capítulo 3, são apresentados os trabalhos relacionados. O Capítulo 4 detalha o procedimento metodológico adotado e como a abordagem utilizada será avaliada. O Capítulo 5 apresenta a solução que será desenvolvida. Por fim, o Capítulo 6 apresenta as considerações finais e o cronograma da pesquisa.

2 FUNDAMENTAÇÃO TEÓRICA

O capítulo a seguir tem como objetivo discutir os fundamentos teóricos do trabalho. Nele serão abordados o significado do TCC e sua relação com a pesquisa científica, além dos problemas enfrentados durante sua concepção. Além disso, serão explorados conceitos que apoiam o estado da arte da IA, iniciando com o significado de seu nome e as possíveis atuações, além das aplicações na área de ML. Serão destacados o estudo de PLN aplicado aos LLMs, somado ao ajuste de textos, servido com técnicas de engenharia de *prompt*, como o *few-Shot prompting* e o *chain-of-Thought prompting*.

2.1 Trabalho de conclusão de curso

O TCC integra o grupo de trabalhos acadêmicos presentes na produção científica e, de acordo com a Associação Brasileira de Normas Técnicas (2024), é constituída por normas específicas e princípios gerais para a elaboração desses trabalhos, visando a apresentação, ao final da produção, para a obtenção do grau e do diploma de Bacharel e/ou Licenciatura. Com isso, dentre os principais tipos de TCCs, o mais comum é utilizar a denominação de monografia para referir-se a um trabalho teórico que se objetiva sobre um determinado assunto (SILVA *et al.*, 2020).

Dada a origem etimológica do termo e sua evolução, convém que a monografia possui dois sentidos: o estrito, que é o tratamento em forma escrita a ser resultado da pesquisa científica, pretendendo apresentar alguma contribuição relevante ao campo, e o sentido lato, que é todo tratamento científico de primeira mão que resulta da pesquisa em si (SALOMON, 2004).

Corrêa *et al.* (2018) conceitua o TCC como um texto acadêmico individual, constituído de uma sistematização de habilidades e conhecimentos adquiridos ao longo de leituras e atividades desenvolvidas na formação do aluno e, principalmente, na pesquisa do trabalho. Como diz Ciribelli (2003), a pesquisa é vista como a realização concreta de uma investigação em consonância com uma metodologia científica. E com essa pesquisa, será desenvolvido um conhecimento científico sólido obtido no processo metodológico, do qual, havendo questões específicas, o aluno pesquisador deve discutir e explicar seu intuito, além de relacioná-lo com outros fatos já conhecidos (PRAÇA, 2015).

Salomon (2004) defende que a tese principal no curso de graduação é criar uma mentalidade científica necessária para formar o profissional de nível superior, e que a relação

entre a monografia e a pesquisa científica se vê como uma decorrente da outra, não havendo a primeira sem a segunda. Baseado no artigo de Ramos (2011), um TCC pode ser estruturado em três partes principais:

- **Parte pré-textual:** Identificação e Resumo;
- **Parte textual:** Introdução, Problema, Hipótese, Justificativa, Objetivos, Metodologia, Cronograma/Fluxograma, Recursos necessários, Resultados, Discussões e Considerações finais (conclusão);
- **Parte referencial:** Anexos, Apêndices e Bibliografia.

A monografia exibe um elemento caracterizador, que é um estudo completo do tema considerado. Tal estudo traz consigo um tratamento reflexivo que, sem ele, a monografia transforma-se em um mero relatório do procedimento de pesquisa; logo, a reflexão é dominante nesse processo (SALOMON, 2004).

Por fim, com tais conceitos em mente, Dias e Silva (2009) complementam que elaborar um trabalho desse escopo exige um planejamento cuidadoso, em que o aluno deve conectar-se a fundo com o que já foi desenvolvido por outros estudiosos, quais foram seus argumentos e descobrir o que não foi alcançado por eles.

2.2 Inteligência artificial

A Inteligência Artificial (IA) apresenta diversas abordagens para a compreensão de seu significado, desde a etimologia até a aplicação prática em diferentes áreas do conhecimento. Jiang *et al.* (2022) introduz a motivação da IA como uma tentativa dos humanos de infundir a inteligência humana em máquinas, e para efetivar o que um computador seria capaz de fazer, Russell e Norvig (2016) categorizam em quatro condutas como requisito para tal: (i) pensar como um ser humano, (ii) agir como um ser humano, (iii) pensar de maneira racional e (iv) agir de maneira racional. Essa classificação evidencia a variedade de pesquisas existentes na área.

Coppin (2004), além de apresentar uma definição primária, caracteriza a IA como a capacidade de utilizar métodos inspirados no comportamento inteligente dos seres humanos e de outros animais para resolver problemas complexos. A partir dessa definição, é possível compreender a intenção das pesquisas ao longo dos anos, ampliando as diretrizes dessa área da Ciência da Computação.

Com tais requisitos e definições, Chen e Yuan (2021) apresentam uma abrangência de técnicas e métodos da IA em comparação com a tecnologia de computadores tradicional,

sendo os principais:

- **Processamento de linguagem natural:** Gama de técnicas computacionais motivadas por teorias para a análise e representação automáticas da linguagem humana. (CAMBRIA; WHITE, 2014);
- **Aprendizado de máquina:** Aplicação de algoritmos e métodos, que permitem a um computador aprender (JONES, 2008). É a capacidade de detectar e extrapolar padrões e se adaptar à novas circunstâncias (RUSSELL; NORVIG, 2016);
- **Visão computacional:** Incorpora evidências neurofisiológicas em modelos computacionais, consistindo no reconhecimento das ações do sentido humano para perceber o mundo (RUSSELL; NORVIG, 2016);
- **Robótica:** Agentes artificiais que operam com robustez através de toda a incerteza em ambientes estocásticos e inacessíveis. São aptos a realizarem tarefas físicas sem interferência humana direta. (COPPIN, 2004).

Devido às possibilidades de aplicações, a IA também é aplicada na educação, como um campo em desenvolvimento. Tavares *et al.* (2020) destaca exemplos notáveis, a citar: sistemas tutores inteligentes, que instruem individualmente os alunos e se adaptam a cada necessidade, ou até mesmo a aplicação de algoritmos de ML, com o intuito de prever novas situações referentes ao desempenho de algum aluno ou classificar a dificuldade de aprendizagem. Mais recentemente, com os LLMs, uma nova era de possibilidades foi trazida à educação. Wang *et al.* (2024) afirmam com relação à capacidade que os grandes modelos possuem, sendo crucial em momentos como a resolução de questões de diversas áreas do conhecimento, a correção de erros ortográficos, gramaticais e até mesmo na criação de material de estudo; tudo isso conduzido por um único *prompt*, na maioria das vezes.

E com esses grandes modelos sendo uma das evoluções tecnológicas mais recentes da área, tarefas são completamente automatizadas com precisão, aliviando a carga de trabalho para todas as partes envolvidas, materializando o potencial da IA para uma personalização em grande escala do ensino e aprendizado (CHIU *et al.*, 2023).

Por meio disso, a IA tem um papel transformador em diversos campos, inclusive na área da educação, transformando a maneira como se ensina. Ela se estende além dos métodos tradicionais de ensino, implementando-os de forma personalizada, adaptando-se aos estilos de aprendizagem, ritmos e preferências de cada estudante, direcionando a um ambiente educacional mais dinâmico e engajando-os na jornada de aprendizagem (WALTER, 2024).

2.3 Aprendizado de máquina

O Aprendizado de Máquina (AM) é uma subárea abrangente da IA que trata de problemas de aprendizagem. Coppin (2004) conceitua como a função de classificar ou prever entradas a partir de um conjunto de dados, tipicamente com uma coleção de dados de treinamento. Assim, o sistema tenta aprender a partir desse conjunto e classificar dados ainda não vistos. Russell e Norvig (2016) completam o raciocínio ao simplificar o agente que realiza tal classificação: o computador, que observa alguns dados, monta um modelo baseado neles e utiliza o modelo como uma hipótese sobre o mundo e um *software* que pode resolver problemas.

A evolução do AM está ligada diretamente à crescente disponibilidade de dados e ao avanço do poder computacional. Dito isso, após o sucesso dos LLMs, os pesquisadores foram atraídos a aplicá-los em diversas áreas, pois estes são, por si, resultado da pesquisa em AM, mais precisamente em Aprendizado Profundo (AP), com os modelos *transformers* (WANG *et al.*, 2025).

O AM possui diversas aplicações no cotidiano, e entre elas, Sen e Dutta (2021) destacam algumas, como: sistemas de negociação de ações, sistemas médicos com classificação de imagens para exames e detecção de doenças, e até mesmo algoritmos preditivos sobre eventos futuros. Um exemplo é a aplicação deles para prever o desempenho futuro de estudantes em instituições de ensino superior, ao combinar notas de anos anteriores, mostrando que a educação é um campo que pode se beneficiar do uso de AM (SEN; DUTTA, 2021). Além disso, com o uso de LLMs, o processo pode ser mais aproveitado e otimizado devido à facilidade com que os modelos conseguem aprender e retornar ao usuário o que foi solicitado, destacando-se nas tarefas, mesmo em *prompts zero-shot* (WANG *et al.*, 2025).

Jones (2008) explica que o AM foca nessas áreas por meio da aplicação de algoritmos e métodos que permitem a um computador aprender, comumente por meio de exemplos advindos de um conjunto de dados cuidadosamente selecionados. Russell e Norvig (2016) determinam os três principais tipos de aprendizagem como sendo: aprendizado supervisionado, aprendizado não supervisionado e aprendizado por reforço. Essas três abordagens possuem aplicações particulares, pois, de acordo com Alpaydin (2021), a escolha desse modelo de aprendizagem é um dos pontos mais críticos do processo, visto que essa decisão guiará boa parte do estudo e influenciará os resultados.

2.3.1 Aprendizado supervisionado

O aprendizado supervisionado trata da ação que o algoritmo de aprendizado (agente) realiza ao observar alguns exemplos de pares de entrada, que seriam aqueles dados em que o algoritmo seria aplicado, e a saída, que seriam seus rótulos. Esse agente aprende a função, recebe um dado e prevê o rótulo apropriado (RUSSELL; NORVIG, 2016).

Sarker (2021) fornece diversos exemplos de aplicação desse modelo de aprendizado, como a detecção de fraudes em cartões de crédito, previsão de disponibilidade de estacionamento, sugestões personalizadas de produtos com base no histórico de compras, entre outros.

2.3.2 Aprendizado não supervisionado

O aprendizado não supervisionado aprende por meio de métodos que não dependem de qualquer intervenção humana (COPPIN, 2004). Enquanto o supervisionado evoca a ideia de que há um professor orientando o processo de aprendizagem, o não supervisionado é ausente dessa ideia, tornando mais difícil o processo de construção de um modelo a partir dos dados (CORD; CUNNINGHAM, 2008). Dito isso, Russell e Norvig (2016) explicam que o agente aprende padrões na entrada, e sua tarefa mais comum é o *clustering*, que trata da detecção de grupos de exemplos de entrada parcialmente úteis.

Um exemplo de aplicação é quando milhões de imagens são adquiridas da *internet*, e um sistema de visão computacional pode identificar um grande grupo de imagens semelhantes que alguém chamaria de “gatos” (RUSSELL; NORVIG, 2016).

2.3.3 Aprendizado por reforço

Kaelbling *et al.* (1996) conceitua o aprendizado por reforço como o problema enfrentado por um agente que deve aprender o comportamento por meio de interações de tentativa e erro com um ambiente dinâmico. O agente aprende a partir de uma série de reforços: recompensas e punições (RUSSELL; NORVIG, 2016). A recompensa é dada quando o modelo performa corretamente, e a punição, quando contrário; ou seja, um reforço positivo e negativo, respectivamente (JONES, 2008). Uma maneira de entender tal conceito é a partir de alguns exemplos:

- **Robô móvel:** Decidir se deve entrar ou não em um cômodo em busca de lixo. Ele toma sua decisão com base no nível de carga atual da bateria e na rapidez e

facilidade com que conseguiu encontrar o carregador, anteriormente (SUTTON *et al.*, 1998).

- **Movimento em um jogo de xadrez:** A escolha é baseada tanto no planejamento — antecipando possíveis respostas e contrarrespostas — quanto em julgamentos imediatos e intuitivos sobre a conveniência de determinadas posições e movimentos (SUTTON *et al.*, 1998).

2.4 Processamento de linguagem natural

O Processamento de Linguagem Natural (PLN) ou *Natural Language Processing* (NLP) é um ramo da IA que teve início nos anos 1950, sendo um campo que evoluiu da fusão entre IA e linguística e, hoje em dia, é um domínio complexo e multidisciplinar que busca processar e extrair significado da linguagem humana (NADKARNI *et al.*, 2011). Chowdhary (2020) conceitua o PLN como uma coleção de técnicas computacionais para análise automática e representação da linguagem humana, motivada pela teoria. No entanto, sabe-se que essa compreensão não é fácil, devido à ambiguidade da linguagem natural; isto é, uma mesma sentença pode ter mais de um significado, podendo, além de ser vaga, não oferecer especificidade em algumas ocasiões (RUSSELL; NORVIG, 2016).

Desde o início de seu estudo, o PLN teve enfoque nas tarefas relacionadas à tradução, recuperação de informações de textos, extração de informações, resumos, respostas de questões, entre outros (CHOWDHARY, 2020). Tantos foram os avanços que, por conta disso, culminou-se na utilização do AP para tornar possível tarefas difíceis de executar, simplesmente utilizando regras e critérios fixos, não exigindo que um programador forneça regras explícitas para cada significado possível; o próprio algoritmo deduz o processo de mapeamento da entrada para a saída. Johri *et al.* (2021). Algumas aplicações práticas incluem tradução automática e geração de textos/diálogos de forma automatizada (CHOWDHARY, 2020).

Atualmente, o PLN é de uma importância incontestável, do qual Goldberg (2017) discute amplamente as aplicações das Redes Neurais (RN), inclusive para o mundo real, como a classificação de documentos, tradução automática, análise de sentimentos, detecção de gramaticalidade, entre outros. Há também assistentes virtuais, como o *Google Assistant*, *Siri*, *Cortana* e *Alexa*, que utilizam técnicas de PLN, presentes no cotidiano dos usuários, sendo integrados casualmente até que, em algum momento, tornem-se um componente facilitador e essencial na gestão da agenda e até mesmo na automação de tarefas das pessoas (TULSHAN; DHAGE,

2018).

Qin *et al.* (2024) adiciona que essas mudanças permitiram que tarefas de PLN fossem impulsionadas pela chegada dos grandes modelos de linguagem, com um paradigma unificado generativo, transformando a maneira com que se trabalha nesse campo. Além disso, até mesmo na educação, a aplicação do PLN foi aprimorada, em especial na universidade, por meio da coleta de *feedback* dos alunos, podendo retornar uma resposta customizada baseada nas necessidades de cada um, mostrando que esses LLMs generativos são adaptáveis a inúmeras aplicações no meio acadêmico, permitindo o desenvolvimento de modelos especializados para tarefas específicas (FUCHS, 2023).

Assim, o PLN mantém sua evolução constante, principalmente com a popularização dos LLMs, capazes de realizar cada vez mais tarefas impressionantes de PLN como a compreensão e a geração da linguagem natural, o impacto para as aplicações práticas atuais e futuras é algo transformador, impactando e mudando a forma como humanos e máquinas interagem entre si (QIN *et al.*, 2024).

2.5 Modelos de linguagem de grande escala

Os Modelos de Linguagem de Grande Escala (LLM) são uma evolução considerável no campo da IA e do PLN. Thirunavukarasu *et al.* (2023) define como sistemas de IA treinados com bilhões de palavras derivadas de artigos, livros e outros conteúdos da internet. Esses modelos computacionais têm a capacidade de entender e gerar a linguagem humana, além da capacidade de prever a probabilidade de sequências de palavras ou gerar novos textos com base em uma determinada entrada (CHANG *et al.*, 2024).

Os LLMs são algoritmos de ML treinados em grandes conjuntos de dados de texto para gerar um conteúdo semelhante ao que humanos fariam, traduzir idiomas e também responder a várias perguntas. Eles representam o mais recente avanço em modelos de linguagem e no campo de PLN (NAVEED *et al.*, 2023). Atualmente, a arquitetura mais utilizada nos LLMs é a arquitetura *transformer*, e Zhao *et al.* (2023) explica que, devido à sua excelente paralelização e capacidade, é possível escalar os modelos para centenas ou milhares de bilhões de parâmetros, permitindo que eles prevejam com precisão a próxima palavra em uma sentença.

Essa abordagem tornou possível que modelos como o *Generative Pre-Trained Transformers (GPT)*, *DeepSeek*, *Llama*, *Gemini*, entre outros, integrassem o cotidiano das pessoas, sendo aplicados em tarefas de sumarização de textos extensos, geração de códigos em diversas

linguagens de programação e até mesmo na detecção de plágio em textos acadêmicos (GAO *et al.*, 2025; YENDURI *et al.*, 2024). Toda essa capacidade de compreensão de texto não é atribuída a um treinamento específico para entender a linguagem como os humanos, mas sim a uma abordagem de aprendizado de associações estatísticas entre palavras, para que se desenvolva a capacidade de prever qual palavra melhor completa uma frase (THIRUNAVUKARASU *et al.*, 2023).

Os LLMs demonstraram uma importância expressiva, onde (THIRUNAVUKARASU *et al.*, 2023) afirma que estes revolucionaram o PLN. Apesar disso, é essencial que haja conhecimento sobre os riscos e desafios do uso, como a geração de textos incorretos e desviados do significado original (alucinações), bem como riscos de instruções que levem o modelo a respostas ofensivas e tendenciosas (ZHAO *et al.*, 2023). Com isso, todo esse potencial deve vir acompanhado do uso responsável e ético por todos os usuários.

2.6 Engenharia de prompt

A Engenharia de prompt é uma técnica recente usada pelos modelos generativos de linguagem com o intuito de extrair sua capacidade máxima, utilizando instruções específicas para a geração de respostas precisas. Heston e Khun (2023) veem como um processo crucial para maximizar os benefícios dos modelos, envolvendo o design de entrada (*prompt*) de uma forma que guie o modelo a produzir a saída desejada. Como a qualidade dos *prompts* afeta diretamente a qualidade das respostas geradas, compreender as nuances da engenharia de *prompt* é vital para criar interações eficazes e significativas com o LLM (EKIN, 2023).

Os *prompts* podem possuir diferentes abordagens e formatos, incluindo uma padronização que permite ao usuário garantir que a saída do LLM siga uma estrutura precisa, utilizando *placeholders* para o conteúdo e preservando a formatação e o *template* fornecido, induzindo o modelo a produzir saídas mais precisas (WHITE *et al.*, 2023). Essa forma de estruturação torna possível guiar o comportamento do modelo e melhorar a qualidade de suas respostas, fornecendo a estrutura e o contexto necessários para que o modelo gere saídas significativas, alinhadas com os objetivos do usuário (GIRAY, 2023).

Um *prompt* pode ser organizado como uma instrução completa, fornecendo contexto e até mesmo um indicador de formato de saída. Marvin *et al.* (2023) destaca os principais elementos em um *prompt*:

- **Instrução:** Entrada de texto específica que guia o comportamento do modelo;

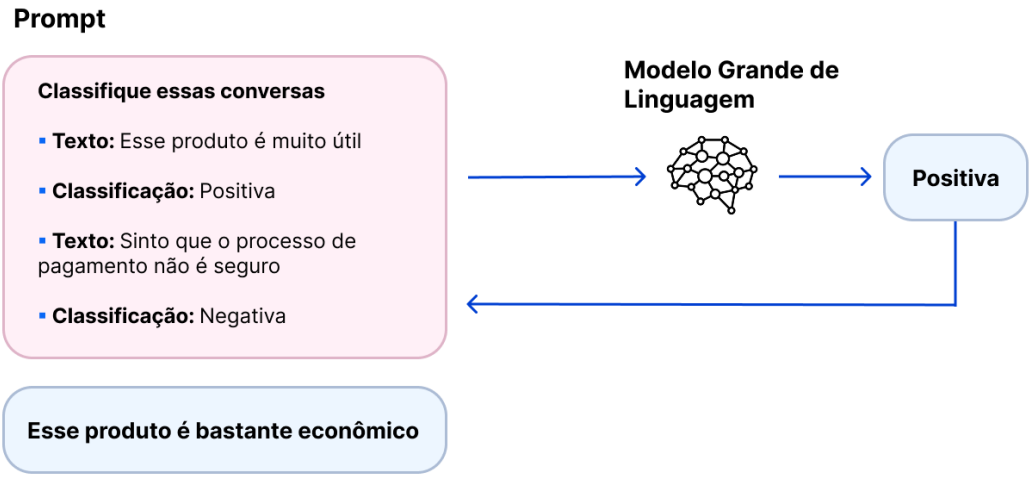
- **Dados de entrada:** Entrada ou pergunta que se deseja que o modelo processe;
- **Contexto:** Forma de fornecer contexto de fundo ao modelo para respostas mais relevantes;
- **Indicador de saída:** Tipo ou formato da saída desejada (parágrafo curto, história de ficção científica).

A partir do exposto, é possível citar algumas das técnicas mais utilizadas em engenharia de *prompt*, incluindo as que estarão presentes nesta pesquisa.

2.6.1 Few-Shot prompting

O *Few-Shot prompting* [1] baseia-se em um pequeno número de exemplos de tarefas resolvidas que são fornecidos como parte da entrada para o modelo treinado (REYNOLDS; MCDONELL, 2021). Em contrapartida ao modelo *zero-shot prompting*, que possui apenas uma pergunta na qual não são incluídos exemplos adicionais, o *few-shot prompting* traz uma técnica de aprendizado com um número mínimo de exemplos.

Figura 1 – Exemplo do *few-shot learning* para classificação de sentimento utilizando métodos baseados em *prompts*.



Fonte: Adaptado de (IBM, s.d.).

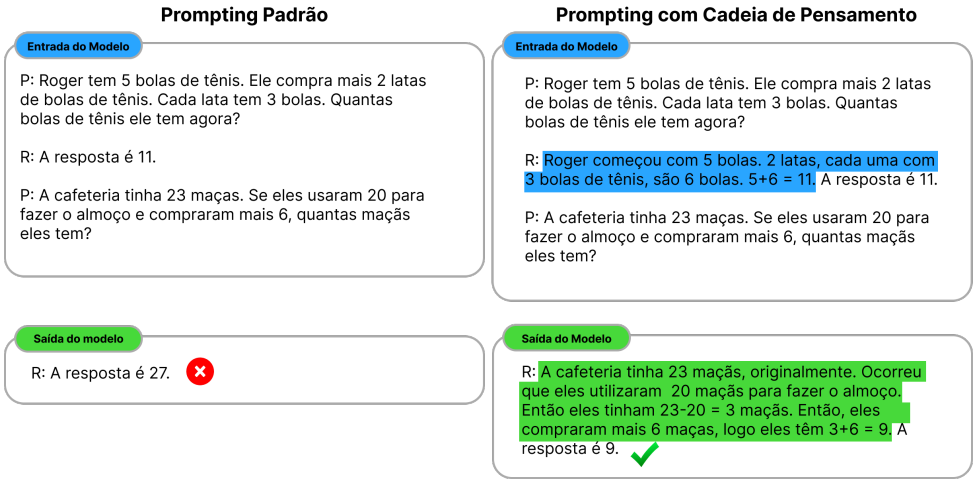
2.6.2 Chain-of-Thought prompting

O *Chain-of-Thought prompting* consiste em uma série de passos de raciocínio intermediários em linguagem natural que levam à saída final de um problema e que podem

[1] https://www.promptingguide.ai/pt/techniques/fewshot

ser utilizados para uma variedade de tarefas, como problemas de matemática, raciocínio de bom senso e manipulação simbólica, além de ser potencialmente aplicável a qualquer tarefa que os humanos possam resolver por meio da linguagem (WEI *et al.*, 2022).

Figura 2 – Exemplo de processo utilizando *chain-of-thought*.



Fonte: Adaptado de (WEI *et al.*, 2022).

3 TRABALHOS RELACIONADOS

Esta seção irá explicitar sobre os trabalhos correlatos, tendo em vista consolidar a compreensão do cenário atual e das contribuições relevantes de outros pesquisadores no trabalho com modelos generativos de linguagem que auxiliam na correção de textos acadêmicos. A análise de seus resultados se deu por meio da análise e correção do conteúdo de acordo com as normas acadêmicas do idioma dos artigos, na construção dos encadeamentos lógicos e rigor metodológico, como também da ferramenta de uso desenvolvida pelos autores e os agentes integrados com LLMs que a compõem.

3.1 Can large language models provide useful feedback on research papers? A large-scale empirical analysis

O trabalho de Liang *et al.* (2024) propõe uma *pipeline* automatizada utilizando o *GPT-4*, recebendo como entrada o texto integral de artigos científicos, a fim de prover comentários úteis, espelhando a estrutura de revisão de importantes periódicos interdisciplinares e conferências, incluindo: importância e novidade, possíveis motivos para aceitação, possíveis motivos para rejeição e sugestões de melhoria.

O *feedback* é organizado em quatro seções: significância e novidade, razões potenciais para aceitação, razões potenciais para rejeição e sugestões para melhoria. O *pipeline* analisa todo o artigo a partir do *Portable Document Format* (PDF) e, em seguida, constrói um *prompt* específico para o artigo com o *GPT-4*. Este *prompt* é criado concatenando as instruções projetadas com o título, o resumo, as legendas das figuras e tabelas do artigo e o restante do texto principal. O *prompt* é então inserido no *GPT-4*, que gera o *feedback* científico em uma única passagem.

Sua metodologia envolveu a realização de uma análise empírica em larga escala, utilizando dois grandes *datasets* compostos por 3.096 artigos aceitos de 15 periódicos da revista *Nature* e 1.709 artigos da *International Conference on Learning Representations* (ICLR), com 6.505 comentários de revisores humanos, incluindo artigos aceitos e rejeitados entre as conferências de 2022 e 2023. A pesquisa envolveu 308 pesquisadores de 110 instituições dos EUA, das áreas de IA e biologia computacional, que avaliaram o *feedback* gerado pelo *GPT-4* em seus próprios artigos.

Para os periódicos da Nature, a sobreposição média foi de 30,85% (*GPT-4* vs.

humano) contra 28,58% (humano vs. humano); para ICLR, foi de 39,23% (*GPT-4* vs. humano) contra 35,25% (humano vs. humano). E quanto aos artigos rejeitados da ICLR, a sobreposição foi de 47,09%, dando a entender que o *feedback* pode ser construtivo para artigos que exigem revisões maiores. Os experimentos também mostraram que os *feedbacks* não são genéricos, mostrando a propensão de identificar questões comuns reconhecidas por múltiplos revisores humanos.

Em resumo, o artigo apresenta uma abordagem inovadora para o *feedback* automatizado sobre artigos científicos, combinando a capacidade dos LLMs de se adaptar a um contexto por meio de grandes dados de treinamento e fornecer respostas satisfatórias. Mesmo com suas limitações, o trabalho serviu, inclusive, como referência para outras abordagens posteriores, como o trabalho de Chamoun *et al.* (2024) e D'Arcy *et al.* (2024).

3.2 Automated focused feedback generation for scientific writing assistance

Em Chamoun *et al.* (2024), é proposta uma ferramenta que fornece comentários acionáveis e sugere revisões ao identificar problemas durante o processo de escrita de artigos científicos. O modelo tem enfoque em parágrafos específicos do artigo e, por vezes, requer acesso completo a ele e à literatura relevante para melhor contexto e precisão em sua saída.

A ferramenta é chamada de *Scientific WrIting Focused Feedback Tool* (SWIF2T), um sistema que alavanca instâncias do *GPT-4* por meio de uma arquitetura multiagente composta por quatro componentes principais: *planner*, *investigator*, *reviewer* e *controller*. O *planner* formula perguntas relevantes e cria um plano passo a passo para os outros componentes. O *investigator* responde às perguntas utilizando informações do próprio *paper* ou da literatura relevante (encontrada na *web*). O *reviewer* utiliza as informações coletadas para identificar uma porção específica do texto que requer revisão, gerando o *feedback* focado e acionável. E o *controller* gerencia o progresso do plano e coordena as ações dos outros modelos, decidindo a sequenciação das etapas ou o rearranjo delas.

A metodologia foi composta pela compilação de um conjunto de dados de teste com revisões por pares escritas por humanos, e um total de 300 unidades foi extraído de *datasets* representativos, incluindo *NLPeer* (DYCKE *et al.*, 2023), *F1000rd* (KUZNETSOV *et al.*, 2022) e *ARIES* (D'ARCY *et al.*, 2024). Esses conjuntos contêm revisões completas de artigos e citações sobre fraquezas em parágrafos específicos. Foi realizado um processo de filtragem com o intuito de garantir a qualidade e relevância do *feedback* e além disso, foi feita uma lista específica com

cinco tipos de fraquezas presentes em artigos, sendo estas: *replicability*, *originality*, *empirical and theoretical soundness*, *meaningful comparison* e *substance*. O *dataset* resultante consistiu em 2.581 parágrafos emparelhados com as revisões escritas por humanos.

Uma análise empírica em larga escala sobre o *feedback* científico gerado pelo *GPT-4* revelou que sua qualidade é comparável à dos revisores humanos. A principal métrica retrospectiva utilizada, a taxa de sobreposição dos pontos levantados, mostrou que o *GPT-4* se sobrepôs aos revisores humanos individuais (30,85% nos periódicos Nature; 39,23% no ICLR) em um grau semelhante ao observado entre dois revisores humanos. O *feedback* do LLM também se provou específico para o artigo, e não genérico. Em um estudo prospectivo, mais da metade dos 308 pesquisadores (57,4%) considerou o *feedback* útil/muito útil, e 82,4% o acharam mais benéfico do que o *feedback* de pelo menos alguns revisores humanos. Embora o *GPT-4* tenda a focar em certos aspectos, como sugestões para "adicionar experimentos em mais conjuntos de dados", os resultados sugerem que o *feedback* de LLM pode complementar a avaliação humana, especialmente na ausência de *feedback* especializado e oportuno.

3.3 MARG: Multi-Agent review generation for scientific papers

O trabalho de D'Arcy *et al.* (2024) propõe o *Multi-Agent Review Generator* (MARG), uma ferramenta multiagente que utiliza várias instâncias do *GPT-4* que se comunicam internamente, buscando automatizar a geração de *feedback* acionável de revisão por pares para artigos científicos, com sugestões e críticas para a melhoria do texto.

Os agentes são especializados com *prompts* construídos de acordo com a função que irão desempenhar, recebendo tarefas adaptadas para diferentes tipos de comentários, como experimentos, clareza e impacto. A arquitetura é composta por um agente do tipo *leader*, que coordena as tarefas e a comunicação entre os agentes, um ou mais *workers*, que são os agentes que receberão um pedaço (*chunk*) do artigo, e, por fim, um ou mais *experts*, que são especializados via *prompting* para aumentar o conhecimento em alguma subtarefa coordenada pelo *leader*.

A partir da definição do MARG, foi proposta uma variante: o *Multi-Agent Review Generator with Specialized Agents* (MARG-S), que aprimora o método de organização da arquitetura.

Portanto, a despeito de algumas limitações, como a demora para a geração da correção, o MARG-S evidencia que a abordagem multiagente pode ser uma alternativa eficaz para lidar com tarefas complexas que exigem raciocínio técnico e uma análise detalhada. Os

resultados apontam que esse tipo de interação entre agentes amplia o potencial da IA na revisão por pares, ao mesmo tempo que abre caminho para futuras melhorias, especialmente no que diz respeito à redução dos custos e ao aumento da qualidade do *feedback* gerado pelos LLMs.

O método comparativo utilizado para a geração dos resultados comparou as duas abordagens, denotando que o MARG-S supera o MARG, dada sua maior robustez.

O *dataset* selecionado veio do corpus *ARIES* (D'ARCY *et al.*, 2024). Esse é um conjunto de edições em artigos científicos corrigidos em resposta a revisões por pares. Um conjunto de 30 artigos do *ARIES* foi usado como conjunto de testes, onde os comentários acionáveis foram extraídos pelo *GPT-4* a partir de revisões humanas reais.

O MARG-S foi avaliado por meio de uma avaliação automatizada e de um estudo com usuários, obtendo bons resultados. No estudo de usuário, o MARG-S reduziu a taxa de comentários genéricos de 60% para 29% em comparação com outros *baselines* usando o *GPT-4*. Foram gerados 3.7 comentários bons por artigo (avaliados pelos usuários). Os comentários gerados foram bastante precisos e específicos, com 71% deles atingindo essa classificação, sendo a maior entre os métodos comparativos. Ao analisar a avaliação automatizada, o MARG-S superou todas as *baselines* em termos de *recall* (considerada a métrica mais importante do trabalho), com 15.84, que seria a fração de comentários humanos alinhados com qualquer comentário gerado. A quantidade de comentários gerados pelo *MARG-S* foi a maior, com 19.8. As versões especializadas do *MARG-S* para impacto e experimentos obtiveram os melhores resultados em comparação com a versão para clareza.

3.4 OpenReviewer: a specialized large language model for generating critical scientific paper reviews

O que se vê no trabalho de Idahl e Ahmadi (2025) é um sistema *open-source* chamado *OpenReviewer*, que gera revisões de alta qualidade para artigos científicos específicos da área de ML e IA. A ferramenta oferece aos autores um *feedback* rápido e construtivo para que os manuscritos sejam enviados com a melhor qualidade possível antes da submissão.

O *OpenReviewer* é um sistema baseado em um LLM *open-source* especializado, chamado *Llama-OpenReviewer-8B*, treinado para retornar as revisões acadêmicas a partir de um vasto *dataset*, com 79.000 revisões de especialistas de importantes conferências de ML.

O conjunto de dados foi obtido pela coleta de 36.000 artigos submetidos e 141.000 revisões da ICLR e da *Conference on Neural Information Processing Systems* (NeurIPS), a partir

das edições de 2022 em diante. Os artigos foram obtidos em formato PDF, na versão mais antiga possível; Ao final de uma etapa de filtragem, apenas 79.000 revisões de alta confiança foram consideradas pelos próprios autores do trabalho e utilizadas para o *fine-tuning* do modelo *LLama-OpenReviewer-8B*.

O *OpenReviewer* emprega um *prompt* que direciona o LLM a agir como um revisor especialista para conferências de IA, seguindo as diretrizes do guia de revisores do ICLR do ano de 2024. O comprimento máximo da janela de contexto do modelo foi de 128.000 *tokens* para que pudesse acomodar textos longos. O processo de treinamento teve duração aproximada de 34 horas, sendo executado em 64 *Graphics Processing Unit*s (GPUs) do modelo *NVIDIA A100*, com 80GB de memória cada.

As avaliações foram baseadas na comparação de revisões geradas com revisões humanas, utilizando um conjunto de teste com 400 artigos retidos e suas revisões do ICLR e NeurIPS. Os resultados demonstram que o *OpenReviewer* produz revisões mais críticas e realistas em comparação com os outros LLMs, correspondendo muito melhor às recomendações dos revisores humanos em 55,5% das revisões geradas. Os LLMs de propósito geral tendem a fornecer recomendações muito positivas, com falta de criticidade, enquanto o *OpenReviewer* correspondeu bem ao *feedback* construtivo. Ao realizar o método de avaliação conhecido como *LLM-as-a-judge*, com o *GPT-4o* como juiz, o *OpenReviewer* obteve taxas de vitória que variaram de 60% contra o *GPT-4o* a 76% contra o *Llama-3.1-70B-Instruct*, indicando um alinhamento maior com as revisões de especialistas humanos.

Mesmo que a precisão da conversão do formato PDF para *Markdown* não seja perfeita, e o tamanho do modelo seja composto por apenas 8 bilhões de parâmetros, os resultados positivos deste trabalho estimulam pesquisas futuras sobre a revisão automatizada de trabalhos acadêmicos, abrindo caminho para impulsionar melhores resultados com o uso de LLMs *open-source*.

3.5 Análise comparativa

No âmbito desta seção, serão abordados de maneira pormenorizada os pontos críticos referentes às métricas e lacunas identificadas nos estudos relacionados até o momento.

O trabalho de Liang *et al.* (2024) aborda a dificuldade em obter *peer reviews* de qualidade e em tempo hábil. O *GPT-4* apresentou taxas médias de sobreposição (*Hit Rate*) de 30,85% para artigos da *Nature* e 39,23% para o ICLR, atingindo 47,09% em artigos rejeitados, o

que indica boa capacidade em detectar falhas evidentes. Além disso, 57,4% dos pesquisadores classificaram o *feedback* como "útil" ou "muito útil". No entanto, o modelo tende a gerar comentários genéricos — como "adicionar experimentos" (2.19 vezes mais frequente que humanos) — e aborda 10.69 vezes menos a novidade do trabalho, revelando limitações em profundidade crítica e especificidade técnica.

O estudo de D'Arcy *et al.* (2024) propõe o MARG-S para mitigar a limitação de contexto dos LLMs e reduzir *feedbacks* genéricos. O sistema produziu, em média, 3.7 comentários "bons" por artigo (2.2 vezes melhor que o *baseline*), com 71% de comentários específicos e apenas 29% genéricos. Em avaliação automática, obteve *Recall* de 15.84, *Precision* de 4.41 e índice *Jaccard* de 3.53, superando os comparativos em até 6.1 pontos. Apesar do avanço em qualidade, apresentou um custo elevado — média de 51.255 *tokens* gerados — e entre 38% e 48% das saídas foram consideradas imprecisas ou de baixa qualidade.

O trabalho de Chamoun *et al.* (2024) propõe o SWIF2T, que melhora a especificidade e a acionabilidade do *feedback* em relação a *baselines* de LLMs. Nas avaliações humanas, obteve *scores* mais altos em Especificidade, Acionabilidade e Utilidade Geral. Nas métricas automáticas, registrou *METEOR* de 20.04, *BLEU@4* de 30.06 e *ROUGE-L* de 20.44, indicando maior consistência com revisões humanas. Contudo, o uso combinado de *GPT-4* e buscas no *Google* gera um custo médio de 7 minutos e 11 segundos por cada exemplo, o que pode introduzir imprecisões na citação de literatura.

No trabalho de Idahl e Ahmadi (2025) é apresentado o *Llama-OpenReviewer-8B*, criado para corrigir o viés positivo de modelos generalistas como o *GPT-4o* e o *Claude-3.5*. O modelo alcançou Pontuação Média de Recomendação de 5.4 (idêntica à humana), *Exact Match* de 55,5% e Erro Absoluto Médio de 0.96, superando o *GPT-4o* (23,8% e 2.34, respectivamente). Também obteve *Win Rate* de 60% na *Review Arena*, indicando maior alinhamento com especialistas. Porém, suas limitações incluem escopo restrito a artigos de ML e IA, e dependência da precisão na conversão de arquivos PDF.

Após uma apresentação de exemplos de trabalhos que exploram a temática de LLMs aplicados a artigos científicos para a geração de *feedback*, agora é possível discernir algumas nuances e distintas contribuições que cada um desses estudos oferece. O presente trabalho inova ao utilizar um modelo *open-source* completamente focado em TCCs em português, optando pelo uso de um LLM *open-source* gratuito, totalmente voltado para o uso de alunos de graduação. Não obstante, o LLM selecionado será ajustado para operar especificamente sobre o texto inserido e,

por meio de configurações adicionais, será aprimorado para retornar uma resposta mais precisa e útil, de acordo com o *prompt* pré-definido na ferramenta e a pré-configuração feita pelo aluno. Além disso, o trabalho será avaliado por sua usabilidade, utilizando a métrica de usabilidade *System Usability Scale* (SUS).

Aprofundando-se na análise comparativa, torna-se evidente que este trabalho se destaca por apresentar uma soma das abordagens e soluções ainda não documentadas no Brasil, diferenciando-se das pesquisas anteriormente discutidas. A Tabela 1 apresenta a comparação entre os estudos.

Pontos analisados:

1. *Feedback* automatizado em trabalhos científicos;
2. Uso de LLM *open-source*;
3. Uso de *prompts especializados*;
4. Arquitetura multiagente;
5. Geração de *feedback* em português brasileiro;

Tabela 1 – Comparação das principais contribuições dos trabalhos relacionados

Autores	**Solução utilizada**	**Métricas**	**1**	**2**	**3**	**4**	**5**
Liang *et al.* (2024)	*Pipeline* automatizada que gera *feedback* estruturado sobre artigos científicos utilizando *GPT-4*.	Taxa de Sobreposição Simkiewicz-Simpson, Índice Jaccard e Coeficiente Sørensen–Dice.	Sim	Não	Sim	Não	Não
Chamoun *et al.* (2024)	Sistema que gera feedback focado, específico e acionável, identificando fraquezas em artigos científicos.	Similaridade Textual (*METEOR, BLEU@4, ROUGE-L*), Precisão do rótulo de aspecto e *F1-Score*.	Sim	Não	Sim	Sim	Não
D'Arcy *et al.* (2024)	Múltiplos agentes de LLM para gerar *feedback* de revisão por pares específico. Útil para artigos científicos longos.	*Recall*, *Precision*, Jaccard, Comentários considerados "Bons"por Artigo e Especificidade de Comentários.	Sim	Não	Sim	Sim	Não
Idahl e Ahmadi (2025)	Abordagem com LLM open-source que gera revisões críticas e estruturadas de artigos de ML/IA.	Correspondência exata da recomendação, Erro absoluto médio, Pontuação Média (Criticidade) e *Win Rate* em *Review Arena*.	Sim	Sim	Sim	Não	Não
Este trabalho	Arquitetura multiagente com LLM *open-source* e *prompts* especializados que geram *feedbacks* de TCCs em português.	Métrica de usabilidade com o SUS e uma *baseline* comparativa entre o *feedback* gerado de LLMs com *zero-shot* e a abordagem *open-source* aprimorada.	Sim	Sim	Sim	Sim	Sim

Fonte: elaborada pelo autor.

4 PROCEDIMENTO METODOLÓGICO

Este capítulo apresenta a metodologia que será utilizada para a construção da ferramenta de revisão automatizada de TCCs, que utilizará uma abordagem multiagentes com LLM *open-source*, apresentando desde o modelo de linguagem selecionado para integrar à ferramenta, as tecnologias que serão utilizadas, a função de cada agente, entre outros. A Seção 4.1 detalha quais serão as tecnologias utilizadas para o desenvolvimento da ferramenta e também do modelo integrado a ela. A Seção 4.2 trata da origem do conjunto de dados e de como ele será adquirido para, posteriormente, ser utilizado pelos modelos presentes nos agentes. A Seção 4.3 apresenta os métodos de avaliação que serão utilizados para medir o desempenho e a utilidade da ferramenta, com destaque para a técnica *System Usability Scale* (SUS), que objetiva garantir a confiabilidade sobre a eficiência do estudo, e a avaliação humana por pesquisadores experientes.

4.1 Tecnologias de implementação da aplicação

O ARAMIS será uma aplicação *web* com sua própria identidade visual prevista, idealizada e prototipada no FIGMA. A implementação será focada em recursos da linguagem de programação *Python*, visando a facilidade existente na integração com bibliotecas e *frameworks* de desenvolvimento *web*, manipulação de dados e requisições. As tecnologias utilizadas na construção do ARAMIS se dividirão em um *Front-End* a ser construído utilizando o *Streamlit*, um *framework* que permite criar uma interface gráfica amigável e intuitiva, que também é facilmente integrável ao *Back-End*, aproveitando-se das chamadas *Hypertext Transfer Protocol* (HTTP), que serão elaboradas com a tecnologia *FastAPI*. Após uma consulta à sua documentação e a percepção adquirida sobre seu alto desempenho em comparação com outras abordagens, como o *Flask*, o *FastAPI* se destaca por sua maior escalabilidade e também gera automaticamente uma documentação com *OpenAPI/Swagger*, o que facilita os testes e as chamadas a *endpoints*, auxiliando no processo de desenvolvimento da ferramenta.

O sistema terá a persistência de dados sustentada pelo *MySQL* durante sua fase de desenvolvimento, em virtude da simplicidade de configuração, realização de testes e armazenamento em banco de dados relacional, visando garantir que as chamadas ao banco retornem corretamente o que foi solicitado. O *MySQL* irá armazenar os dados dos usuários cadastrados na plataforma, o histórico de conversação do usuário — permitindo consulta futura —, entre outros. Ciente de suas limitações, ele irá armazenar apenas dados mais simples, como os já citados, não

tendo previsão de alocar além do escopo definido neste trabalho.

Todas as abordagens citadas anteriormente serão utilizadas em sua versão mais recente, pelo menos até o momento em que este trabalho foi escrito, com a linguagem de programação sendo inteiramente em *Python 3.13*, assim como as bibliotecas previstas para uso; o *Front-End* usará *Streamlit 1.50.0*, o *Back-End FastAPI 0.118.0* e, para o *database* o *phpMyAdmin 5.2.3* com *MySQL*, para fechar a construção do *Minumum Viable Product* (MVP) da ferramenta proposta.

4.2 Conjunto de dados

Para a especialização dos agentes que compõem o ARAMIS, será utilizado um conjunto de dados composto por TCCs aprovados de alunos de graduação da UFC, inicialmente do campus de Crateús, e especificamente da área da computação, definida no escopo do trabalho, entre os períodos de 2018 a 2024, e caso haja uma quantidade insuficiente, será expandido para os outros campi presentes na universidade, abrangendo o período de 2020 a 2024. Esses trabalhos serão coletados a partir do Repositório Institucional da universidade e de trabalhos disponibilizados de forma voluntária por seus autores, com a prévia autorização para uso acadêmico.

O conjunto de TCCs que será extraído no formato *PDF* passará por um pré-processamento antes de ser utilizado pelo modelo. Esse conjunto será extraído e convertido por meio da biblioteca *Marker* para o formato *Markdown*, um formato que mantêm uma estrutura com marcadores melhor do que apenas o texto sequencialmente definido, preservando a estrutura hierárquica do documento. A qualidade da conversão será avaliada manualmente pela separação das seções, de acordo com a organização padrão dos TCCs. A tendência será aderir a uma abordagem mais precisa e, além disso, que lide bem com tabelas e fórmulas matemáticas, o que é o caso do *Marker*, já que tabelas e fórmulas são altamente presentes em TCCs da computação.

A escolha por um corpus de TCCs da área supracitada justifica-se pela afinidade com o objetivo deste trabalho e pela delimitação de escopo definida no planejamento, visando a maior precisão nas revisões. O texto será extraído dos TCCs por meio de um *pipeline*, onde, para os agentes de correção gramatical, de encadeamento lógico e rigor metodológico, a extração incluirá o conteúdo inteiro do texto, preservando a estrutura de capítulos e seções, com destaque à marcação dessas partes do texto, com o intuito de conferir um peso maior ao rigor de sua análise, mas ignorando ou removendo algumas informações não pertinentes, como o nome dos

autores, orientadores, dedicatória, agradecimentos, entre outros.

4.3 Avaliação da ferramenta

A avaliação deste trabalho será realizada ao avaliar as revisões geradas pelo LLM e a usabilidade da ferramenta produzida. A primeira será uma *baseline* para comparar as revisões geradas pelo modelo que será utilizado neste trabalho com outros modelos, sejam eles proprietários ou não, para garantir a comprovação de que o modelo adaptado operou melhor em determinadas circunstâncias em comparação com os outros, aplicando-os a mesma entrada e conferindo se a saída corresponde a uma revisão de acordo com o objetivo citado na Seção 1.3. A segunda será utilizando o SUS, desenvolvido por Brooke (1995), uma métrica escolhida por ser amplamente reconhecida para medir avaliações subjetivas de usabilidade em uma ampla gama de contextos, inclusive na avaliação de Sistemas de Informação (SI). Além disso, conforme Padrini-Andrade *et al.* (2019), essa possibilidade de avaliar diversos tipos de contextos deve-se ao fato de que o SUS é tecnologicamente agnóstico, ou seja, possui uma abordagem neutra e aberta. O usuário deverá avaliar a usabilidade do ARAMIS imediatamente após a conclusão das tarefas no sistema, para que não haja discussões ou reflexões que possam enviesar a opinião. O questionário SUS consiste em 10 perguntas, funcionando de acordo com a Figura 3, onde os usuários responderão em uma escala *Likert* que varia de 1 a 5, onde 1 representa *"Discordo Completamente"* e 5 representa *"Concordo Completamente"*. A Figura 3 ilustra as perguntas que o questionário contém no processo de avaliação e exibe a escala de avaliação.

Figura 3 – Exemplo do questionário SUS que será utilizado

Questionário System Usability Scale	**Discordo Completamente**				**Concordo Completamente**
1. Acho que gostaria de usar este sistema com frequência.	1	2	3	4	5
2. Achei o produto desnecessariamente complexo	1	2	3	4	5
3. Achei o sistema fácil de usar	1	2	3	4	5
4. Acho que precisaria do suporte de um técnico para poder usar este sistema	1	2	3	4	5
5. Achei que as várias funções deste sistema estavam bem integradas	1	2	3	4	5
6. Achei que havia muita inconsistência neste sistema	1	2	3	4	5
7. Imagino que a maioria das pessoas aprenderia a usar este sistema muito rapidamente	1	2	3	4	5
8. Achei o sistema muito complicado de usar	1	2	3	4	5
9. Eu me senti muito confiante ao usar o sistema	1	2	3	4	5
10. Precisei aprender muitas coisas antes de começar a usar esse sistema.	1	2	3	4	5

Adaptado de (BROOKE, 1995).

4.3.1 Cálculo do SUS

Após a conclusão do questionário, o SUS avalia a métrica por meio de um cálculo para consolidar o resultado, que varia de 0 a 100. O método utilizado para a pontuação resultará em um número único que compõe a usabilidade geral do sistema avaliado. Para consolidar esse cálculo, ele seguirá a abordagem de seu artigo de origem em Brooke (1995), que envolve a seguinte metodologia: os itens ímpares (Afirmações Positivas) tem sua nota de atribuição subtraída por 1, enquanto para os itens pares (Afirmações Negativas), o cálculo é feito subtraindo o valor 5 da nota de atribuição. Após a conclusão, todas as pontuações das dez perguntas são somadas e o resultado é multiplicado por 2,5. O processo de cálculo do SUS está exemplificado na Figura 4.

Brooke (1995), em seu artigo, não definiu uma escala concreta de pontuação para determinar se a usabilidade seria boa ou ruim. Por isso, este trabalho seguirá a interpretação

defendida por Bangor *et al.* (2009), que resolve essa lacuna do trabalho de Brooke. A abordagem apresentada é a seguinte:

- **Excelente**: Pontuação entre 90 e 100;
- **Bom**: Pontuação entre 80 e 89;
- **Aceitável**: Pontuação entre 70 e 79;
- **Precisa Melhorar**: Pontuação entre 60 e 69;
- **Ruim**: Pontuação abaixo de 60.

Figura 4 – Exemplo do cálculo realizado na avaliação SUS

PERGUNTAS	PARTICIPANTE
Pergunta 1	3 (3-1 = 2)
Pergunta 2	5 (5-5 = 0)
Pergunta 3	4 (4-1 = 3)
Pergunta 4	1 (5-1 = 4)
Pergunta 5	3 (3-1 = 2)
Pergunta 6	3 (5-3 = 2)
Pergunta 7	4 (4-1 = 3)
Pergunta 8	1 (5-1 = 4)
Pergunta 9	4 (4-1 = 3)
Pergunta 10	1 (5-1 = 4)
(2+0+3+4+2+2+3+4+3+4) * 2.5 = 67.5	

Adaptado de (BARBOZA, 2019).

$$SUS = 2,5 \times \left[\sum_{i=1}^{10} f(x_i) \right] \tag{4.1}$$

onde:

- Para itens **ímpares** (1, 3, 5, 7, 9): $f(x_i) = x_i - 1$
- Para itens **pares** (2, 4, 6, 8, 10): $f(x_i) = 5 - x_i$
- x_i é a nota do usuário para cada item (escala de 1 a 5)

5 PROPOSTA

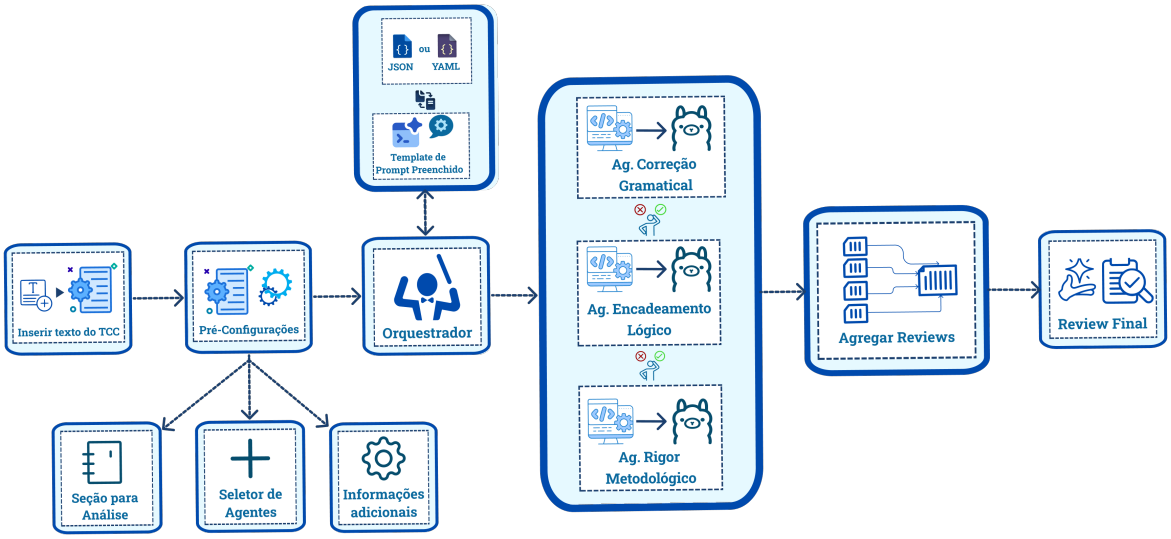
Este capítulo apresenta a proposta de materialização da ferramenta ARAMIS, incluindo sua concepção, arquitetura e a forma como os dados serão tratados. A Seção 5.1 contém uma breve explicação sobre a origem do nome da ferramenta, além de um apanhado geral do fluxo de funcionamento do ARAMIS, incluindo todas as suas etapas de operação com os trechos de TCCs inseridos. A Seção 5.2 discorre sobre a escolha do modelo *open-source* mais adequado para realizar o treinamento e uso na ferramenta, onde a seleção será baseada em algumas métricas de desempenho do modelo em si. A Seção 5.3 apresenta a justificativa sobre o tamanho do modelo que será utilizado e onde ele pode ser encontrado. A Seção 5.4 detalha o processo de operação e funcionamento dos agentes do ARAMIS. A Seção 5.5 discute como será o desenvolvimento dos *prompts* a serem aplicados em cada agente da ferramenta.

5.1 ARAMIS

O sistema a ser concebido neste trabalho é chamado de ARAMIS e seu acrônimo faz breve alusão a um personagem central de Os Três Mosqueteiros (DUMAS, 1844). O ARAMIS nada mais é do que uma plataforma composta por agentes especializados que permite a interação entre o usuário e uma espécie de *chat* que receberá o texto do TCC do aluno. Para que funcione, a aplicação será composta por agentes integrados a um LLM *open-source* voltados exclusivamente para tratar de assuntos ligados ao TCC em questão. O ARAMIS terá agentes que atuarão sobre níveis que se complementam — correção gramatical, encadeamento lógico e rigor metodológico, com atribuições internas entre esses três agentes —, ativando-se a partir da configuração do usuário, com o objetivo de fornecer *feedbacks* precisos e úteis sobre o texto do trabalho em desenvolvimento.

5.1.1 Arquitetura e fluxo do ARAMIS

Figura 5 – A figura apresenta o fluxograma do ARAMIS, detalhando suas principais etapas.



Fonte: elaborada pelo autor.

Inicialmente, para acessar o ARAMIS, o usuário deverá ter um cadastro na plataforma. Feito isso, ao fazer o login, será apresentada a interface de interação com a ferramenta, onde haverá quatro opções de acesso: a página inicial, contendo uma breve apresentação do ARAMIS; o acesso principal ao chat de inserção do texto claro do TCC do aluno; a seção de informações úteis sobre a ferramenta, que exibirá uma descrição geral dos componentes que fazem parte do ARAMIS; e a área de gerenciamento do perfil, onde o usuário poderá gerenciar algumas opções, como a foto de perfil, nome, *email*, entre outros.

Tratando-se do funcionamento da seção de chat, que é um dos objetivos principais deste trabalho, será exibido um tópico de conversa em que o usuário irá inserir o texto do TCC, seja apenas uma seção ou ele inteiro, pois o LLM utilizado para a correção deverá ter uma janela de contexto de aproximadamente 128.000 *tokens*, o que permite abranger documentos com centenas de páginas, assegurando, assim, uma linha de raciocínio coesa sobre o *prompt* recebido. Nesta tela, haverá uma parte da interface com opções de pré-configuração sobre o comportamento do modelo, o que auxiliará no formato de resposta que o ARAMIS deverá retornar, direcionando-o a devolver uma resposta precisa e estruturada. Serão algumas das opções de pré-configuração:

- Selecionar uma ou mais seções a que se refere o texto inserido pelo usuário;
- Definir entre um e três agentes que podem ser acionados para gerar o *feedback*;

- Inserir informações adicionais para auxiliar no direcionamento do LLM, como o título do trabalho e a área específica do conhecimento.

Na terceira etapa, baseada nas informações preenchidas na pré-configuração, uma função chamada Orquestrador será responsável por receber os valores temporariamente armazenados nas variáveis padrão dos *prompts*, citadas na Seção 5.5. Dessa forma, as lacunas das variáveis de um ou mais *templates* dos *prompts* especializados, contendo as informações da etapa anterior, estarão completas. Feito isso, os *templates* preenchidos serão convertidos para um formato de notação de arquivo de configuração, como o formato *JavaScript Object Notation* (JSON) ou *YAML Ain't Markup Language* (YAML), com o intuito de facilitar a leitura do *prompt* que será enviado aos modelos integrados aos agentes.

O Orquestrador, que tem seu nome inerente à sua função, ainda operará sobre a decisão de ativação dos agentes, recebendo a escolha do usuário na pré-configuração e ativando os itens selecionados. A atividade exercida por cada agente será de maneira sequencial, isto é, caso os três agentes sejam acionados, o próximo só começará a operar após a conclusão da operação do agente anterior, devido a existência de apenas um LLM integrado à ferramenta, funcionando para apenas uma consulta por vez; e, se houver apenas um agente acionado, os outros não deverão operar, permanecendo inativos durante toda a rodada do fluxo do ARAMIS.

Por fim, o resultado das revisões será sequenciado em um único arquivo JSON (caso apenas um agente seja acionado, haverá apenas uma revisão, também em apenas um arquivo), separado pela resposta de cada agente, e quando estiverem unificadas, serão exibidas na interface ao usuário, separadas pelo título de cada agente a que pertencem, permitindo a visualização da correção gerada pelo modelo de linguagem utilizado no ARAMIS.

5.2 Escolha do LLM open-source

A decisão de utilizar um modelo de linguagem de código aberto se deu pela junção de alguns fatores que favorecem essa escolha, como a permissividade que essa abordagem traz, em contrapartida à maioria dos modelos proprietários fechados, como o *GPT-4, Gemini, Copilot*, entre outros. Um modelo *open-source* possui a vantagem de ter seu código-fonte aberto e disponível para que se possa usar, examinar e modificar. O ARAMIS será uma plataforma com esse tipo de modelo integrado aos seus agentes, pois, dessa forma, é possível controlar seu comportamento com maior liberdade e adaptá-lo para as necessidades específicas e inerentes às revisões de um TCC em português. Ademais, ao optar por esse paradigma, haverá a possibilidade

de modificar alguns de seus parâmetros internos, caso necessário, o que permite direcionar o modelo em uma especialidade única, como a geração de revisões de TCCs em português. Subramanian *et al.* (2025) exprime que a escolha de modelos menores e focados em tarefas precisas é um fator predominante para um melhor desempenho geral na tarefa destinada, onde podem até mesmo superar os grandes, que possuem uma maior quantidade de parâmetros. Outro ponto que será analisado é a ausência da necessidade de licenças, a fim de não depender de pagamentos em dinheiro, garantindo liberdade e autonomia no treinamento e testes do modelo.

5.3 Tamanho do modelo

A escolha por um LLM essencialmente *open-source* se dará por meio de uma pesquisa aplicada, levando em conta o tamanho do modelo para aplicá-lo em um contexto específico como o deste trabalho, a capacidade de rodar localmente em um contexto offline e também por razões de segurança ou até mesmo uma futura disponibilização para *download*. Devido a isso, serão realizados testes locais com alguns modelos de geração de texto de até 120B de parâmetros, dado o bom equilíbrio entre desempenho e custo computacional, somado à análise local em repositórios públicos como o Hugging Face[1], que hospeda modelos da classe *meta-llama*, *deepseek-ai*, entre outros, e reúne métricas importantes para auxiliar na escolha do LLM mais adequado. Para este trabalho, será considerada a capacidade do modelo de processar e gerar texto. Para determinar o modelo mais adequado ao que é proposto, será montada uma *baseline* com o modelo *open-source* escolhido e um modelo proprietário, realizando testes comparativos entre os dois, considerando métricas de desempenho avaliadas por revisores, custo computacional e qualidade linguística das respostas geradas. Serão levadas em conta as restrições de hardware existentes nas máquinas de teste, o que poderá limitar a capacidade do modelo de rodar localmente ou de ser hospedado em servidores com menor poder computacional. Não obstante, a ferramenta poderá ser utilizada pelos usuários alunos interessados em melhorar a escrita de seus trabalhos.

5.4 Função dos agentes do ARAMIS

O ARAMIS, como uma ferramenta multiagente, será composto por um módulo de agentes, que nada mais são do que funções em código fonte comum, onde a lógica utilizada

[1] https://huggingface.co/

dependerá de um orquestrador programado para decidir quais agentes serão utilizados naquela rodada de execução do programa, ou seja, de acordo com o fluxo do ARAMIS, exibido na Figura 5. Dessa forma, dependendo da pré-configuração do usuário, o orquestrador decidirá qual agente será considerado para aquela função direcionada, quando necessária, recebendo uma chamada composta por um corpo de informações passado na requisição.

O ARAMIS conterá um orquestrador que controlará três agentes principais, onde cada um será integrado em instâncias semelhantes do mesmo LLM *open-source*, realizando as seguintes tarefas:

- **Agente 1: Correção gramatical**

  Será o agente menos robusto, organizado com um *prompt* estruturado, utilizando técnicas de Engenharia de Prompt, como as explicadas na Seção 2.6 , voltado especificamente para identificar e listar erros ortográficos e possíveis equívocos de acentuação que o texto inserido pode apresentar, orientado para não renovar ou alterar o estilo de escrita do autor do trabalho.

- **Agente 2: Encadeamento lógico**

  Será o agente preparado para lidar com a coerência entre as sentenças e seções presentes no trabalho. Com isso, a previsão é que seja realizada uma análise do micro (sentenças adjacentes/parágrafos) e do macro (ordem e relação entre seções) pelo LLM. Também será utilizado um *prompt* próprio especializado com técnicas de Engenharia de Prompt, com o intuito de estruturar e aprimorar a resposta gerada pelo modelo. Após isso, o agente será testado, e estando adaptado, será integrado ao *Back-End* do ARAMIS.

- **Agente 3: Rigor metodológico**

  O agente de Rigor Metodológico é o mais elaborado que o ARAMIS possuirá. A partir dessa mentalidade, esse agente, assim como os outros, também conterá um *prompt* próprio direcionado, que utilizará técnicas estruturadas por etapas e será testado com abordagens como o *Few-Shot Prompting*. Esse processo receberá uma seleção de TCCs como fonte de conhecimento, para que o LLM consiga avaliar automaticamente a metodologia e retornar revisões sobre a objetividade e o rigor do que foi escrito. Após isso, o agente será testado e quanto estiver adaptado, será integrado ao *Back-End* do ARAMIS.

5.5 Modelagem dos prompts

Neste estudo, o processo de modelagem dos *prompts* para a geração de revisões será pensado inteiramente para atuar unicamente sobre cada agente, ou seja, haverá um *prompt* especializado para cada um, extraindo os pormenores previstos para um bom *feedback* e mostrando como o ARAMIS pensará na resposta.

Os *prompts* serão estruturados de maneira a receber orientações explícitas sobre quem são, o papel a desempenhar, os detalhes a se atentar e o tipo de resposta a retornar. Além disso, os *prompts* serão dinâmicos, nos quais haverá variáveis (*placeholders*) que o sistema preencherá automaticamente com informações adquiridas em etapas anteriores à ativação dos *prompts*. Essas variáveis conterão, por exemplo, o texto da seção submetida, a área específica do trabalho e o tipo de avaliação desejada. Essas variáveis estarão dentro de chaves, onde serão automaticamente substituídas no momento da execução da LLM, conforme as escolhas do usuário (esse conceito é esclarecido na Subseção 5.1.1). Existirão variáveis padronizadas em todos os *prompts*, pois todos seguirão uma estrutura semelhante, com a diferença do direcionamento legado a cada um deles. Abaixo, detalhamos quais serão essas variáveis que estarão entre chaves para serem substituídas no momento da entrada do modelo:

- *secao_desejada*: Indica nos *prompts* qual(is) seções serão abrangidas para análise, podendo ser apenas uma ou até mesmo todas as seções. Ela ajudará o modelo a inferir apenas sobre as seções selecionadas;
- *titulo_tcc*: Contém o título do trabalho do aluno. Desta forma, o LLM saberá especificamente o tema que está tratando e as informações possivelmente atreladas a ele, adquirindo um contexto preciso;
- *area_conhecimento_tcc*: Representa a área de conhecimento a qual o TCC está atrelado, por exemplo, IA, AM, grafos, entre outros. Essa variável ajudará o LLM a entender por que esse assunto está sendo escrito com aquele conteúdo e daquela maneira;
- *nivel_rigor_modelo*: Compõe o valor correspondente à opção da seleção dentro da ferramenta, definindo o nível de rigor que o modelo deve avaliar aquele trecho do TCC do aluno. Essa variável servirá para apoiar o controle do teor do LLM de acordo com o desejo do usuário;
- *informacoes_adicionais*: Fornece a opção de inserir um texto claro com alguma informação adicional, pois isso ajudará a contextualizar melhor o LLM para

fornecer feedbacks mais direcionados, podendo acrescentar nuances que antes não seriam abordadas.

Esse design específico de *prompt* tem como intenção explorar a capacidade do LLM de capturar padrões complexos, como a objetividade e o rigor científico. Esse enfoque é especialmente útil para os modelos de linguagem, que possuem um forte potencial de generalização e podem identificar padrões globais em uma única leitura dos textos.

6 CONSIDERAÇÕES FINAIS E CRONOGRAMA

Durante toda a concepção deste trabalho, foram estabelecidos os fundamentos para definir o objetivo principal deste trabalho, que é o desenvolvimento de uma ferramenta integrada com um LLM *open-source*, baseada em uma arquitetura multiagente para gerar revisões em TCCs. Ao longo da pesquisa, foi possível levantar a fundamentação teórica, mapear os trabalhos relacionados e definir o procedimento metodológico e a proposta do ARAMIS. Também foi cumprido o papel de definir uma arquitetura dividida em módulos, com a função de cada agente, o pipeline pretendido de extração e processamento de dados utilizando o Marker, o *baseline* de avaliação, o projeto do procedimento metodológico do modelo *open-source* descrito, para especializar o modelo de acordo a exigência dos agentes, e também o protocolo de avaliação do ARAMIS baseado no SUS.

A seguir, é descrito o cronograma planejado com os passos que serão seguidos para materializar a conclusão desta pesquisa. Logo em seguida, são definidas cada uma das atividades, tal como pode ser visualizado através da Tabela 2. ()

Tabela 2 – Cronograma das próximas atividades

Atividades	**Out**	**Nov**	**Dez**	**Jan**
Extração e tratamento dos TCCs do Repositório da UFC	X	X		
Elaboração dos Prompts		X		
Implementação da Plataforma		X	X	
Testes de avaliação do uso da plataforma			X	
Escrita do TCC II				X

Fonte: elaborada pelo autor.

REFERÊNCIAS

ALPAYDIN, E. **Machine learning**. [S.l.]: MIT press, 2021.

Associação Brasileira de Normas Técnicas. **NBR 14724: Informação e documentação – Trabalhos acadêmicos – Apresentação**. 2024. Rio de Janeiro: ABNT. 4. ed.

BANGOR, A.; KORTUM, P.; MILLER, J. Determining what individual sus scores mean: Adding an adjective rating scale. **Journal of usability studies**, Usability Professionals' Association Bloomingdale, IL, v. 4, n. 3, p. 114–123, 2009.

BARBOZA, A. **Medindo a usabilidade do seu produto com System Usability Scale (SUS)**. 2019. Medium. Acesso em: 11 de outubro de 2025. Disponível em: <https://medium.com/design-contaazul/medindo-a-usabilidade-do-seu-produto-com-system-usability-scale-sus-3956612d9229>.

BROOKE, J. Sus: A quick and dirty usability scale. **Usability Eval. Ind.**, v. 189, 11 1995.

CAMBRIA, E.; WHITE, B. Jumping nlp curves: A review of natural language processing research [review article]. **IEEE Computational Intelligence Magazine**, v. 9, n. 2, p. 48–57, 2014.

CARBONI, R. M.; NOGUEIRA, V. d. O. Facilidades e dificuldades na elaboração de trabalhos de conclusão de curso. **ConScientiae Saúde**, v. 3, p. 65–72, jan. 2008. Disponível em: <https://periodicos.uninove.br/saude/article/view/321>.

CHAMOUN, E.; SCHLICHKTRULL, M.; VLACHOS, A. Automated focused feedback generation for scientific writing assistance. **arXiv preprint arXiv:2405.20477**, 2024.

CHANG, Y.; WANG, X.; WANG, J.; WU, Y.; YANG, L.; ZHU, K.; CHEN, H.; YI, X.; WANG, C.; WANG, Y.; YE, W.; ZHANG, Y.; CHANG, Y.; YU, P. S.; YANG, Q.; XIE, X. A survey on evaluation of large language models. **ACM Trans. Intell. Syst. Technol.**, Association for Computing Machinery, New York, NY, USA, v. 15, n. 3, mar. 2024. ISSN 2157-6904. Disponível em: <https://doi.org/10.1145/3641289>.

CHEN, G.; YUAN, Q. Application and existing problems of computer network technology in the field of artificial intelligence. In: **2021 2nd International Conference on Artificial Intelligence and Computer Engineering (ICAICE)**. [S.l.: s.n.], 2021. p. 139–142.

CHIU, T. K.; XIA, Q.; ZHOU, X.; CHAI, C. S.; CHENG, M. Systematic literature review on opportunities, challenges, and future research recommendations of artificial intelligence in education. **Computers and Education: Artificial Intelligence**, v. 4, p. 100118, 2023. ISSN 2666-920X. Disponível em: <https://www.sciencedirect.com/science/article/pii/S2666920X2200073X>.

CHOWDHARY, K. R. Natural language processing. In: ____. **Fundamentals of Artificial Intelligence**. New Delhi: Springer India, 2020. p. 603–649. ISBN 978-81-322-3972-7. Disponível em: <https://doi.org/10.1007/978-81-322-3972-7_19>.

CIRIBELLI, M. C. **Como elaborar uma dissertação de mestrado através da pesquisa científica**. [S.l.]: 7Letras, 2003.

COPPIN, B. **Artificial Intelligence Illuminated**. [S.l.]: Jones and Bartlett Publishers, 2004.

CORD, M.; CUNNINGHAM, P. **Machine learning techniques for multimedia: case studies on organization and retrieval**. [S.l.]: Springer Science & Business Media, 2008.

CORRÊA, E. J.; VASCONCELOS, M.; SOUZA, M. d. L. Iniciação à metodologia: Trabalho de conclusão de curso. **Belo Horizonte: NESCON UFMG**, 2018.

D'ARCY, M.; HOPE, T.; BIRNBAUM, L.; DOWNEY, D. Marg: Multi-agent review generation for scientific papers. **arXiv preprint arXiv:2401.04259**, 2024.

D'ARCY, M.; ROSS, A.; BRANSOM, E.; KUEHL, B.; BRAGG, J.; HOPE, T.; DOWNEY, D. ARIES: A corpus of scientific paper edits made in response to peer reviews. In: KU, L.-W.; MARTINS, A.; SRIKUMAR, V. (Ed.). **Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)**. Bangkok, Thailand: Association for Computational Linguistics, 2024. p. 6985–7001. Disponível em: <https://aclanthology.org/2024.acl-long.377/>.

DIAS, D. d. S.; SILVA, M. F. d. **Como escrever uma monografia**. [S.l.]: Universidade Federal do Rio de Janeiro, 2009.

DUMAS, A. **Os Três Mosqueteiros**. 1. ed. [S.l.]: Clássicos Zahar, 1844.

DYCKE, N.; KUZNETSOV, I.; GUREVYCH, I. NLPeer: A unified resource for the computational study of peer review. In: ROGERS, A.; BOYD-GRABER, J.; OKAZAKI, N. (Ed.). **Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)**. Toronto, Canada: Association for Computational Linguistics, 2023. p. 5049–5073. Disponível em: <https://aclanthology.org/2023.acl-long.277/>.

EKIN, S. Prompt engineering for chatgpt: a quick guide to techniques, tips, and best practices. **Authorea Preprints**, Authorea, 2023.

FAGUNDES, C. V.; LUCE, M. B.; ESPINAR, S. R. O desempenho acadêmico como indicador de qualidade da transição ensino médio-educação superior. **Ensaio: Avaliação e Políticas Públicas em Educação**, Fundação CESGRANRIO, v. 22, n. 84, p. 635–669, Jul 2014. ISSN 0104-4036. Disponível em: <https://doi.org/10.1590/S0104-40362014000300004>.

FUCHS, K. Exploring the opportunities and challenges of nlp models in higher education: is chat gpt a blessing or a curse? **Frontiers in Education**, Volume 8 - 2023, 2023. ISSN 2504-284X. Disponível em: <https://www.frontiersin.org/journals/education/articles/10.3389/feduc.2023.1166682>.

GAO, T.; JIN, J.; KE, Z. T.; MORYOUSSEF, G. A comparison of deepseek and other llms. **arXiv preprint arXiv:2502.03688**, 2025.

GIRAY, L. Prompt engineering with chatgpt: a guide for academic writers. **Annals of biomedical engineering**, Springer, v. 51, n. 12, p. 2629–2633, 2023.

GOLDBERG, Y. **Neural network methods in natural language processing**. [S.l.]: Morgan & Claypool Publishers, 2017.

HESTON, T. F.; KHUN, C. Prompt engineering in medical education. **International Medical Education**, MDPI, v. 2, n. 3, p. 198–205, 2023.

IBM. **What is few-shot prompting?** s.d. <https://www.ibm.com/think/topics/few-shot-prompting>. Acesso em: 24 ago. 2025.

IDAHL, M.; AHMADI, Z. OpenReviewer: A specialized large language model for generating critical scientific paper reviews. In: DZIRI, N.; REN, S. X.; DIAO, S. (Ed.). **Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (System Demonstrations)**. Albuquerque, New Mexico: Association for Computational Linguistics, 2025. p. 550–562. ISBN 979-8-89176-191-9. Disponível em: <https://aclanthology.org/2025.naacl-demo.44/>.

JIANG, Y.; LI, X.; LUO, H.; YIN, S.; KAYNAK, O. Quo vadis artificial intelligence? **Discover Artificial Intelligence**, Springer, v. 2, n. 1, p. 4, 2022.

JOHRI, P.; KHATRI, S. K.; AL-TAANI, A.; SABHARWAL, M.; SUVANOV, S.; CHAUHAN, A. Natural language processing: History, evolution, application, and future work. In: ____. [S.l.: s.n.], 2021. p. 365–375. ISBN 978-981-15-9711-4.

JONES, M. T. **Artificial intelligence: a systems approach**. [S.l.]: Laxmi Publications, Ltd., 2008.

KAELBLING, L. P.; LITTMAN, M. L.; MOORE, A. W. Reinforcement learning: A survey. **Journal of artificial intelligence research**, v. 4, p. 237–285, 1996.

KUZNETSOV, I.; BUCHMANN, J.; EICHLER, M.; GUREVYCH, I. Revise and resubmit: An intertextual model of text-based collaboration in peer review. **Computational Linguistics**, v. 48, n. 4, p. 949–986, 12 2022. ISSN 0891-2017. Disponível em: <https://doi.org/10.1162/coli_a_00455>.

LECUN, Y.; BENGIO, Y.; HINTON, G. Deep learning. **nature**, Nature Publishing Group UK London, v. 521, n. 7553, p. 436–444, 2015.

LIANG, W.; ZHANG, Y.; CAO, H.; WANG, B.; DING, D. Y.; YANG, X.; VODRAHALLI, K.; HE, S.; SMITH, D. S.; YIN, Y. *et al.* Can large language models provide useful feedback on research papers? a large-scale empirical analysis. **NEJM AI**, Massachusetts Medical Society, v. 1, n. 8, p. AIoa2400196, 2024.

LUNSFORD, A. A.; LUNSFORD, K. J. "mistakes are a fact of life": A national comparative study. **College Composition and Communication**, National Council of Teachers of English, v. 59, n. 4, p. 781–806, 2008. ISSN 0010096X. Disponível em: <http://www.jstor.org/stable/20457033>.

MARVIN, G.; HELLEN, N.; JJINGO, D.; NAKATUMBA-NABENDE, J. Prompt engineering in large language models. In: SPRINGER. **International conference on data intelligence and cognitive informatics**. [S.l.], 2023. p. 387–402.

MATOS, J. d. C.; FREITAS, T. d. O trabalho de conclusão de curso na visão do aluno: Desafios e ganhos. **Revista Científica Multidisciplinar Núcleo do Conhecimento**, v. 7, n. 10, p. 174–183, out. 2020. ISSN 2448-0959. Disponível em: <https://www.nucleodoconhecimento.com.br/educacao/desafios-e-ganhos>. Acesso em: 13.06.2025.

MEDEIROS, B. C.; SILVA, R. C. L. d.; ROCHA, F. A. F. d.; DANJOUR, M. F. Dificuldades do processo de orientação em trabalhos de conclusão de curso (tcc): Um estudo com os docentes do curso de administração de uma instituição privada de ensino superior. **HOLOS**, v. 5, p. 242–255, out. 2015. Disponível em: <https://www2.ifrn.edu.br/ojs/index.php/HOLOS/article/view/1011>. Acesso em: 13.06.2025.

MONTENEGRO, S. Dificuldades de estudantes de graduação na elaboração do tcc: O desafio da pesquisa. **Diálogos e Perspectivas Interventivas**, v. 5, n. 2, p. e21295, 2024. Disponível em: <https://revistas.uneb.br/index.php/dialogos/article/view/21295>. Acesso em: 13.06.2025.

NADKARNI, P. M.; OHNO-MACHADO, L.; CHAPMAN, W. W. Natural language processing: an introduction. **Journal of the American Medical Informatics Association**, BMJ Group BMA House, Tavistock Square, London, WC1H 9JR, v. 18, n. 5, p. 544–551, 2011.

NAVEED, H.; KHAN, A. U.; QIU, S.; SAQIB, M.; ANWAR, S.; USMAN, M.; AKHTAR, N.; BARNES, N.; MIAN, A. A comprehensive overview of large language models. **ACM Transactions on Intelligent Systems and Technology**, ACM New York, NY, 2023.

OECD. **PISA 2022 Results (Volume I): The State of Learning and Equity in Education**. Paris, 2023. Disponível em: <https://doi.org/10.1787/53f23881-en>.

PADRINI-ANDRADE, L.; BALDA, R. d. C. X.; ARECO, K. C. N.; BANDIERA-PAIVA, P.; NUNES, M. d. V.; MARBA, S. T. M.; CARVALHO, W. B. d.; RUGOLO, L. M. S. d. S.; ALMEIDA, J. H. C. d.; PROCIANOY, R. S. *et al.* Evaluation of usability of a neonatal health information system according to the user's perception. **Revista Paulista de Pediatria**, Sociedade de Pediatria de São Paulo, v. 37, n. 1, p. 90–96, Jan 2019. ISSN 0103-0582. Disponível em: <https://doi.org/10.1590/1984-0462/;2019;37;1;00019>.

PEREZ, M. S.; BOENAVIDES, W. M. Os limites para a revisão do texto literário a partir dos conceitos de autoria e estilo de bakhtin. **Bakhtiniana: Revista de Estudos do Discurso**, LAEL/PUC-SP (Programa de Estudos Pós-Graduados em Linguística Aplicada e Estudos da Linguagem da Pontifícia Universidade Católica de São Paulo), v. 12, n. 1, p. 113–130, Jan 2017. ISSN 2176-4573. Disponível em: <https://doi.org/10.1590/2176-457325830>.

PRAÇA, F. S. G. Metodologia da pesquisa científica: organização estrutural e os desafios para redigir o trabalho de conclusão. **Revista Eletrônica "Diálogos Acadêmicos**, v. 8, n. 1, p. 72–87, 2015.

QIN, L.; CHEN, Q.; FENG, X.; WU, Y.; ZHANG, Y.; LI, Y.; LI, M.; CHE, W.; YU, P. **Large Language Models Meet NLP: A Survey**. 2024.

RAMOS, I. M. L. Orientações gerais do tcc. **São Paulo (SP): Centro Paula Souza**, 2011.

REYNOLDS, L.; MCDONELL, K. Prompt programming for large language models: Beyond the few-shot paradigm. **CoRR**, abs/2102.07350, 2021. Disponível em: <https://arxiv.org/abs/2102.07350>.

RUSSELL, S. J.; NORVIG, P. **Artificial Intelligence: A Modern Approach**. 4. ed. Upper Saddle River, NJ: Pearson, 2016. Terceira edição revisada e ampliada. ISBN 978-0136042594.

SALOMON, D. V. **Como fazer uma monografia**. [S.l.]: Martins Fontes, 2004.

SARKER, I. H. Machine learning: Algorithms, real-world applications and research directions. **SN computer science**, Springer, v. 2, n. 3, p. 160, 2021.

SEN, R. S. J.; DUTTA, A. **Machine Learning - Algorithms, Models and Applications**. IntechOpen, 2021. ISSN 2633-1403. ISBN 9781839694851. Disponível em: <http://dx.doi.org/10.5772/intechopen.94615>.

SILVA, D. F. da; FOGGIATO, A. A.; NETO, J. L. T.; PARREIRAS, S. O. **Manual prático para elaboração de trabalhos de conclusão de curso**. [S.l.]: Editora Blucher, 2020.

SRIVARSHA, N.; NITHIN, G.; HARIHARAN, S.; DASH, B. B.; CHOWDHURY, S.; PATRA, S. S. Auto text correction using nlp techniques. In: **2025 6th International Conference on Mobile Computing and Sustainable Informatics (ICMCSI)**. [S.l.: s.n.], 2025. p. 590–594.

SUBRAMANIAN, S.; ELANGO, V.; GUNGOR, M. **Small Language Models (SLMs) Can Still Pack a Punch: A survey**. 2025. Disponível em: <https://arxiv.org/abs/2501.05465>.

SUTTON, R. S.; BARTO, A. G. *et al.* **Reinforcement learning: An introduction**. [S.l.]: MIT press Cambridge, 1998. v. 1.

TAVARES, L. A.; MEIRA, M. C.; AMARAL, S. F. d. Inteligência artificial na educação: Survey / artificial intelligence in education: Survey. **Brazilian Journal of Development**, v. 6, n. 7, p. 48699–48714, Jul. 2020. Disponível em: <https://ojs.brazilianjournals.com.br/ojs/index.php/BRJD/article/view/13539>.

THIRUNAVUKARASU, A. J.; TING, D. S. J.; ELANGOVAN, K.; GUTIERREZ, L.; TAN, T. F.; TING, D. S. W. Large language models in medicine. **Nature medicine**, Nature Publishing Group US New York, v. 29, n. 8, p. 1930–1940, 2023.

TULSHAN, A. S.; DHAGE, S. N. Survey on virtual assistant: Google assistant, siri, cortana, alexa. In: SPRINGER. **International symposium on signal processing and intelligent recognition systems**. [S.l.], 2018. p. 190–201.

WALTER, Y. Embracing the future of artificial intelligence in the classroom: the relevance of ai literacy, prompt engineering, and critical thinking in modern education. **International Journal of Educational Technology in Higher Education**, Springer, v. 21, n. 1, p. 15, 2024.

WANG, S.; HUANG, J.; CHEN, Z.; SONG, Y.; TANG, W.; MAO, H.; FAN, W.; LIU, H.; LIU, X.; YIN, D. *et al.* Graph machine learning in the era of large language models (llms). **ACM Transactions on Intelligent Systems and Technology**, ACM New York, NY, v. 16, n. 5, p. 1–40, 2025.

WANG, S.; XU, T.; LI, H.; ZHANG, C.; LIANG, J.; TANG, J.; YU, P. S.; WEN, Q. Large language models for education: A survey and outlook. **arXiv preprint arXiv:2403.18105**, 2024.

WEI, J.; WANG, X.; SCHUURMANS, D.; BOSMA, M.; XIA, F.; CHI, E.; LE, Q. V.; ZHOU, D. *et al.* Chain-of-thought prompting elicits reasoning in large language models. **Advances in neural information processing systems**, v. 35, p. 24824–24837, 2022.

WHITE, J.; FU, Q.; HAYS, S.; SANDBORN, M.; OLEA, C.; GILBERT, H.; ELNASHAR, A.; SPENCER-SMITH, J.; SCHMIDT, D. C. A prompt pattern catalog to enhance prompt engineering with chatgpt. **arXiv preprint arXiv:2302.11382**, 2023.

YENDURI, G.; RAMALINGAM, M.; SELVI, G. C.; SUPRIYA, Y.; SRIVASTAVA, G.; MADDIKUNTA, P. K. R.; RAJ, G. D.; JHAVERI, R. H.; PRABADEVI, B.; WANG, W.; VASILAKOS, A. V.; GADEKALLU, T. R. Gpt (generative pre-trained transformer)— a comprehensive review on enabling technologies, potential applications, emerging challenges, and future directions. **IEEE Access**, v. 12, p. 54608–54649, 2024.

ZHANG, M.; ZHANG, Y. The study on nlp-based semantic analysis technology to improve the accuracy of english translation. In: **2023 IEEE International Conference on Control, Electronics and Computer Technology (ICCECT)**. [S.l.: s.n.], 2023. p. 1107–1112.

ZHAO, W. X.; ZHOU, K.; LI, J.; TANG, T.; WANG, X.; HOU, Y.; MIN, Y.; ZHANG, B.; ZHANG, J.; DONG, Z. *et al.* A survey of large language models. **arXiv preprint arXiv:2303.18223**, v. 1, n. 2, 2023.